



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

DD/MM/2026

Análisis y diseño de sistema 2

Escuela de Ingeniería de Ciencias Y Sistemas

NOMBRE DEL TUTOR

UNIDAD #:1

Patrones de diseño estructurales

Anuncios Importantes



Anuncio 1 (Ingresa los anuncios importantes como lo sean, cortos, entregas, practicas, proyectos, horarios de calificación, etc.)



Anuncio 2



Anuncio 3



Anuncio 4



Anuncio 5

Agenda



Dudas proyecto fase 1



Teoría



Caso práctico

COMPETENCIA(S) QUE DESARROLLAREMOS



Identifica patrones de diseño por medio del análisis de las diferentes necesidades para resolver un problema específico que se adapta a la solución propuesta por el patrón seleccionado.



VALOR DE LA SEMANA

- **Disciplina:** es la capacidad de mantener hábitos de estudio, organización y esfuerzo constante, siguiendo normas y objetivos para alcanzar metas académicas y personales efectivamente.

¿DUDAS DEL
PROYECTO FASE 1?

PATRONES DE DISEÑO ESTRUCTURALES



PATRÓN DE DISEÑO ESTRUCTURAL

PATRÓN DE DISEÑO
ESTRUCTURAL

Los patrones estructurales explican cómo ensamblar objetos y clases en estructuras más grandes, a la vez que se mantiene la flexibilidad y eficiencia de estas estructuras.

Analogía

Compara los patrones estructurales con un juego de construcción , como LEGO. Cada patrón te da una manera específica de conectar piezas (clases y objetos) para construir algo complejo de forma organizada y reutilizable.





ADAPTER



ADAPTER

Adapter es un patrón de diseño estructural que permite la colaboración entre objetos con interfaces incompatibles.

Estructura del Patrón

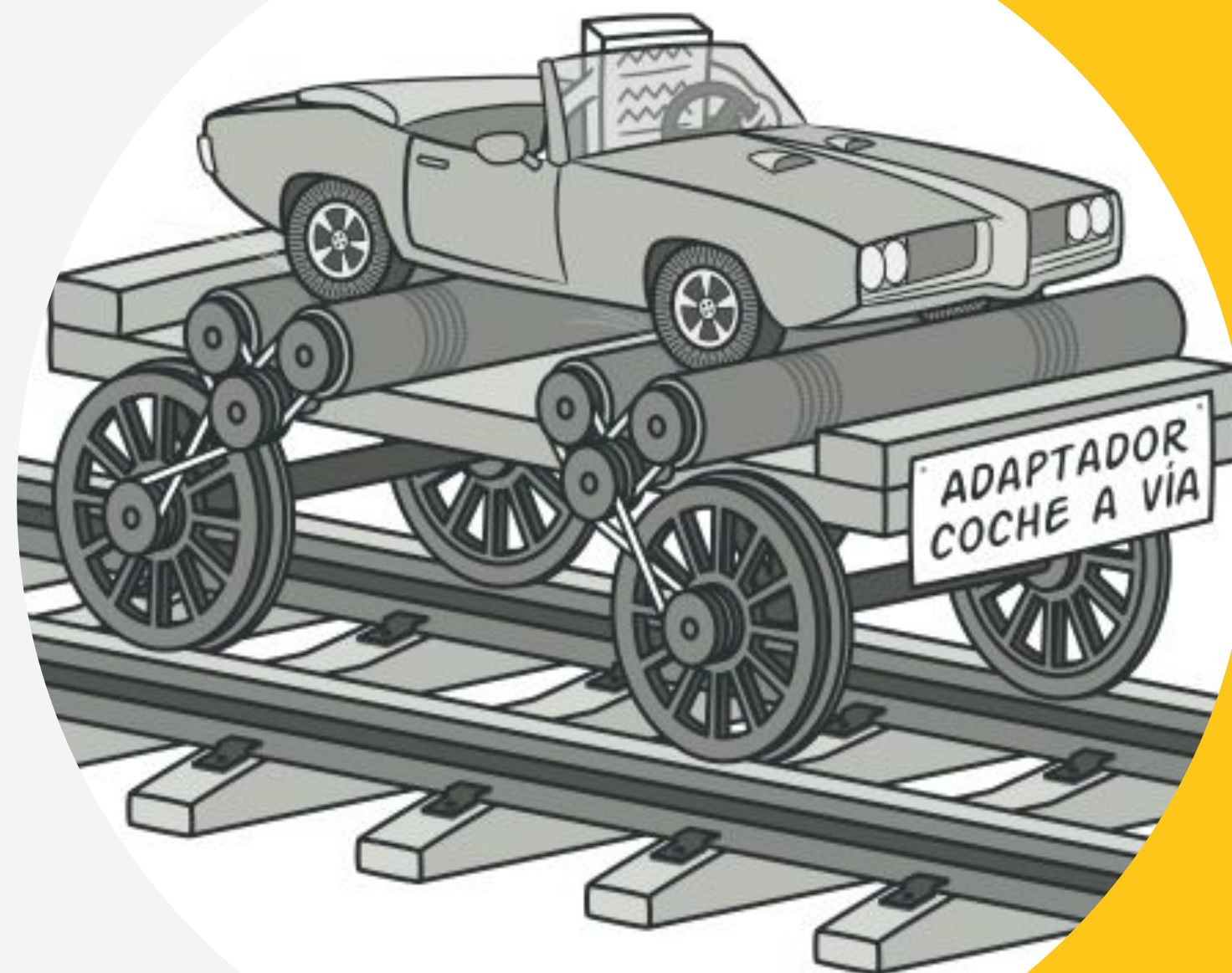
La estructura del Adapter generalmente involucra cuatro componentes clave:

Client (Cliente): La clase que colabora con los objetos que tienen la interfaz de destino. El cliente es el que no puede usar directamente la clase

- **Target (Interfaz Objetivo):** La interfaz que el Client espera. Es la que define el contrato que el Client necesita para funcionar.

- **Adaptee (Adaptado):** La clase con la interfaz incompatible que necesitas usar. Es la clase existente y a menudo de terceros

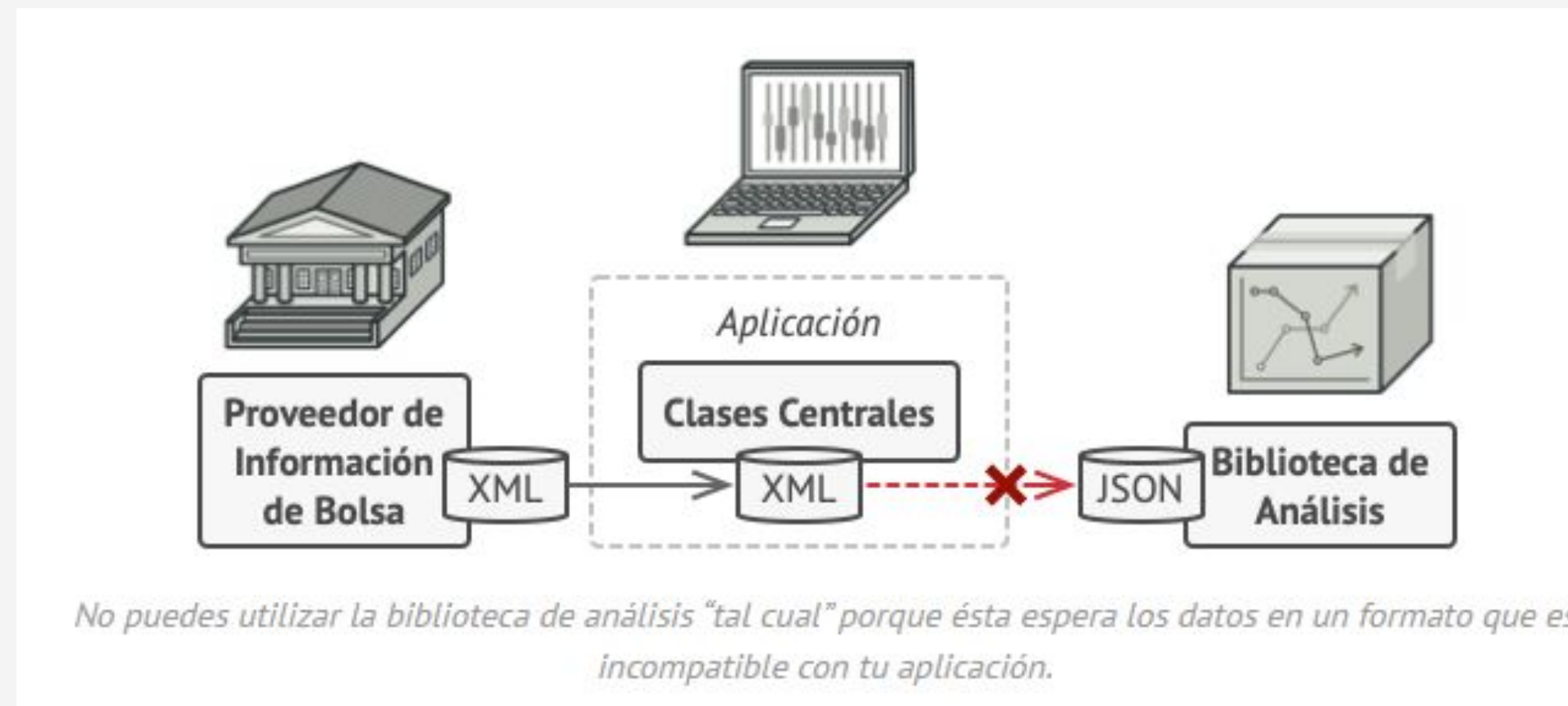
- **Adapter (Adaptador):** La clase que hace la magia. Implementa la interfaz Target y contiene una instancia del Adaptee. En sus métodos, delega las llamadas al Adaptee subyacente, traduciendo las peticiones del Target a las del Adaptee.



PROBLEMA

Imagina que estás creando una aplicación de monitoreo del mercado de valores. La aplicación descarga la información de bolsa desde varias fuentes en formato XML para presentarla al usuario con bonitos gráficos y diagramas.

En cierto momento, decides mejorar la aplicación integrando una inteligente biblioteca de análisis de una tercera persona. Pero hay una trampa: la biblioteca de análisis solo funciona con datos en formato JSON.



Podrías cambiar la biblioteca para que funcione con XML. Sin embargo, esto podría descomponer parte del código existente que depende de la biblioteca. Y, lo que es peor, podrías no tener siquiera acceso al código fuente de la biblioteca, lo que hace imposible esta solución.

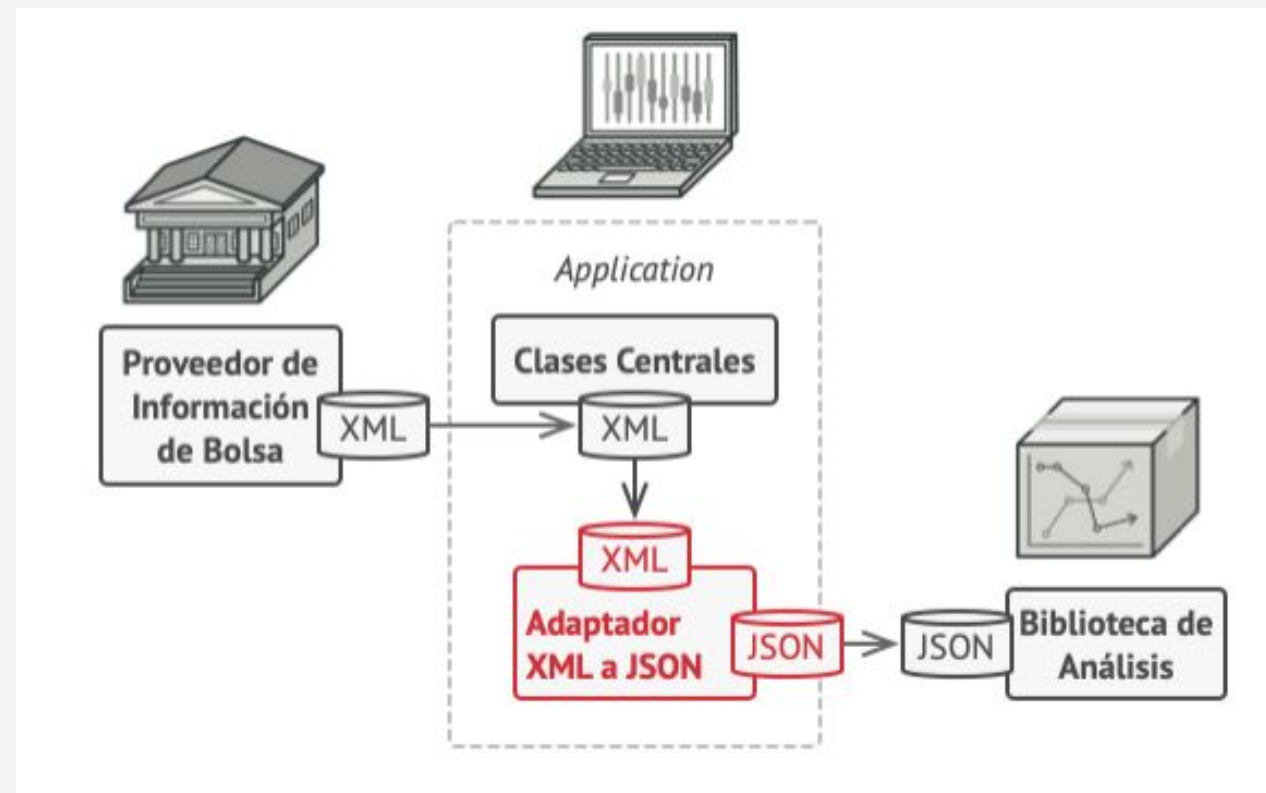
SOLUCIÓN...

Puedes crear un adaptador. Se trata de un objeto especial que convierte la interfaz de un objeto, de forma que otro objeto pueda comprenderla. Un adaptador envuelve uno de los objetos para esconder la complejidad de la conversión que tiene lugar tras bambalinas. El objeto envuelto ni siquiera es consciente de la existencia del adaptador. Por ejemplo, puedes envolver un objeto que opera con metros y kilómetros con un adaptador que convierte todos los datos al sistema anglosajón, es decir, pies y millas.

Los adaptadores no solo convierten datos a varios formatos, sino que también ayudan a objetos con distintas interfaces a colaborar. Funciona así:

1. El adaptador obtiene una interfaz compatible con uno de los objetos existentes.
2. Utilizando esta interfaz, el objeto existente puede invocar con seguridad los métodos del adaptador.
3. Al recibir una llamada, el adaptador pasa la solicitud al segundo objeto, pero en un formato y orden que ese segundo objeto espera.

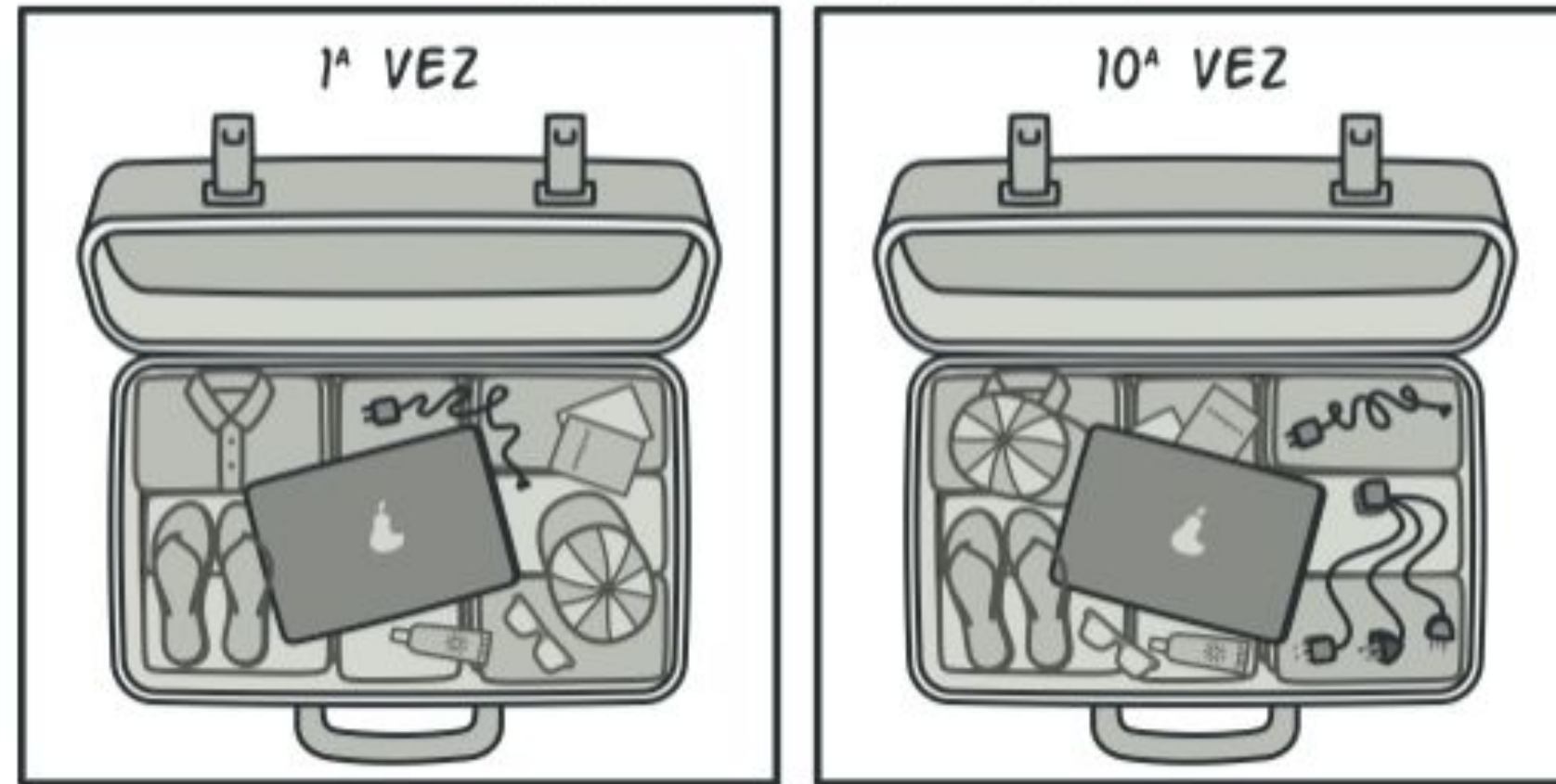
En ocasiones se puede incluso crear un adaptador de dos direcciones que pueda convertir las llamadas en ambos sentidos.



Regresemos a nuestra aplicación del mercado de valores. Para resolver el dilema de los formatos incompatibles, puedes crear adaptadores de XML a JSON para cada clase de la biblioteca de análisis con la que trabaje tu código directamente. Después ajustas tu código para que se comuniquen con la biblioteca únicamente a través de estos adaptadores. Cuando un adaptador recibe una llamada, traduce los datos XML entrantes a una estructura JSON y pasa la llamada a los métodos adecuados de un objeto de análisis envuelto.

REAL

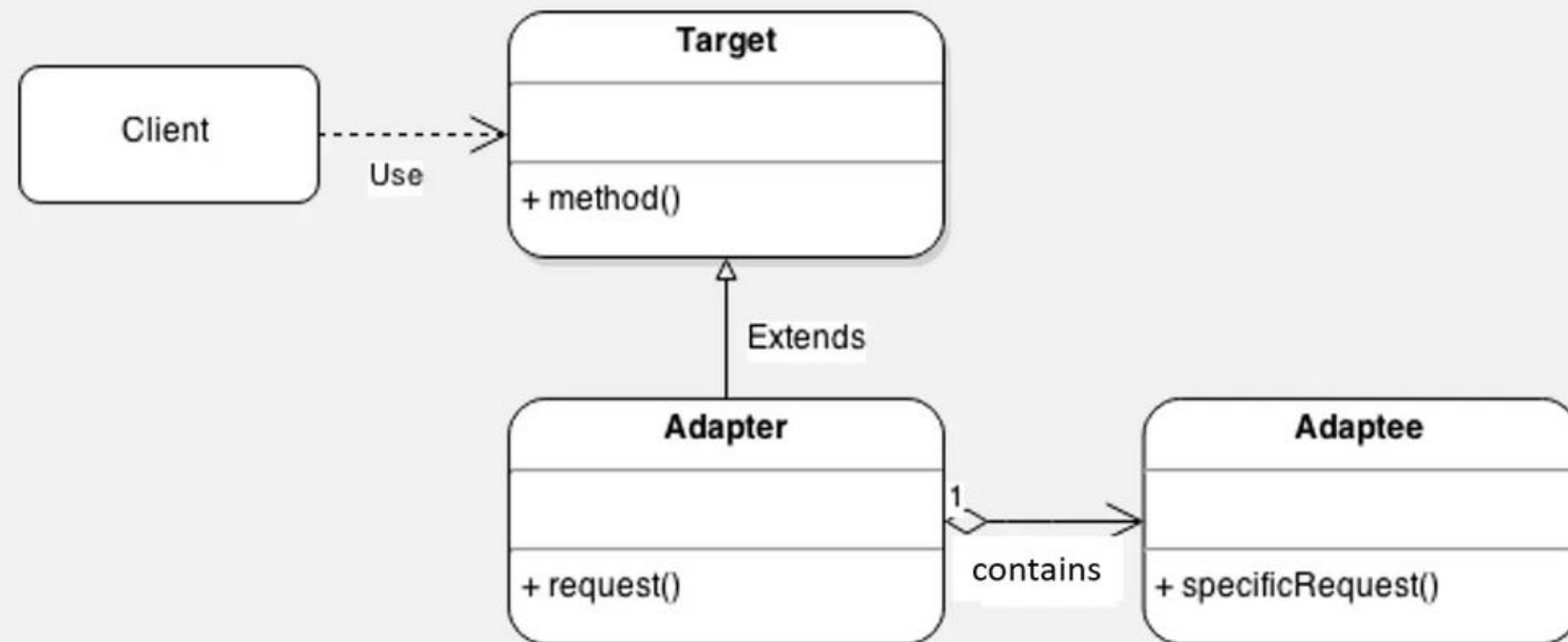
VIAJAR AL EXTRANJERO



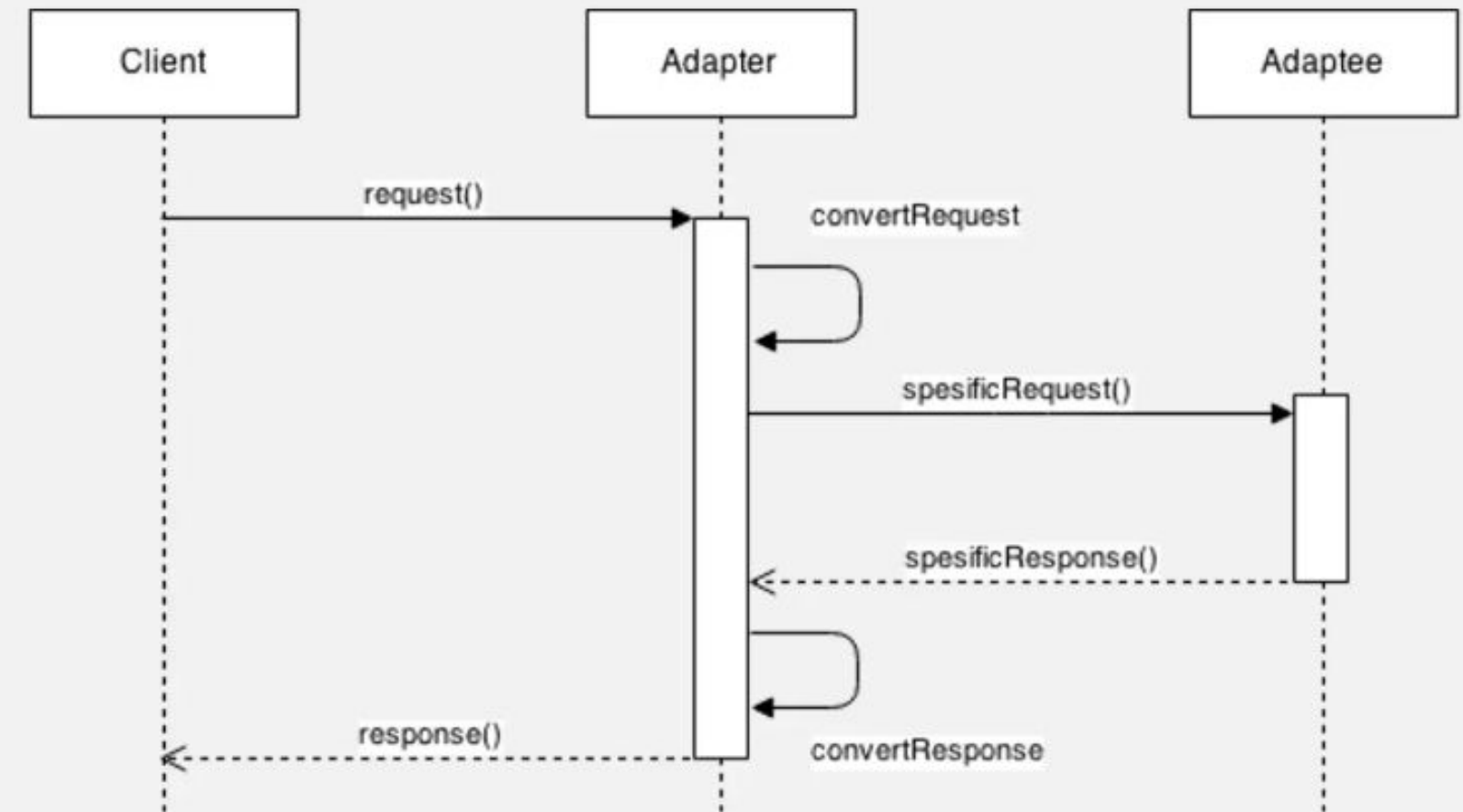
Una maleta antes y después de un viaje al extranjero.

Cuando viajas de Europa a Estados Unidos por primera vez, puede ser que te lleves una sorpresa cuando intentes cargar tu computadora portátil. Los tipos de enchufe son diferentes en cada país, por lo que un enchufe español no sirve en Estados Unidos. El problema puede solucionarse utilizando un adaptador que incluya el enchufe americano y el europeo.

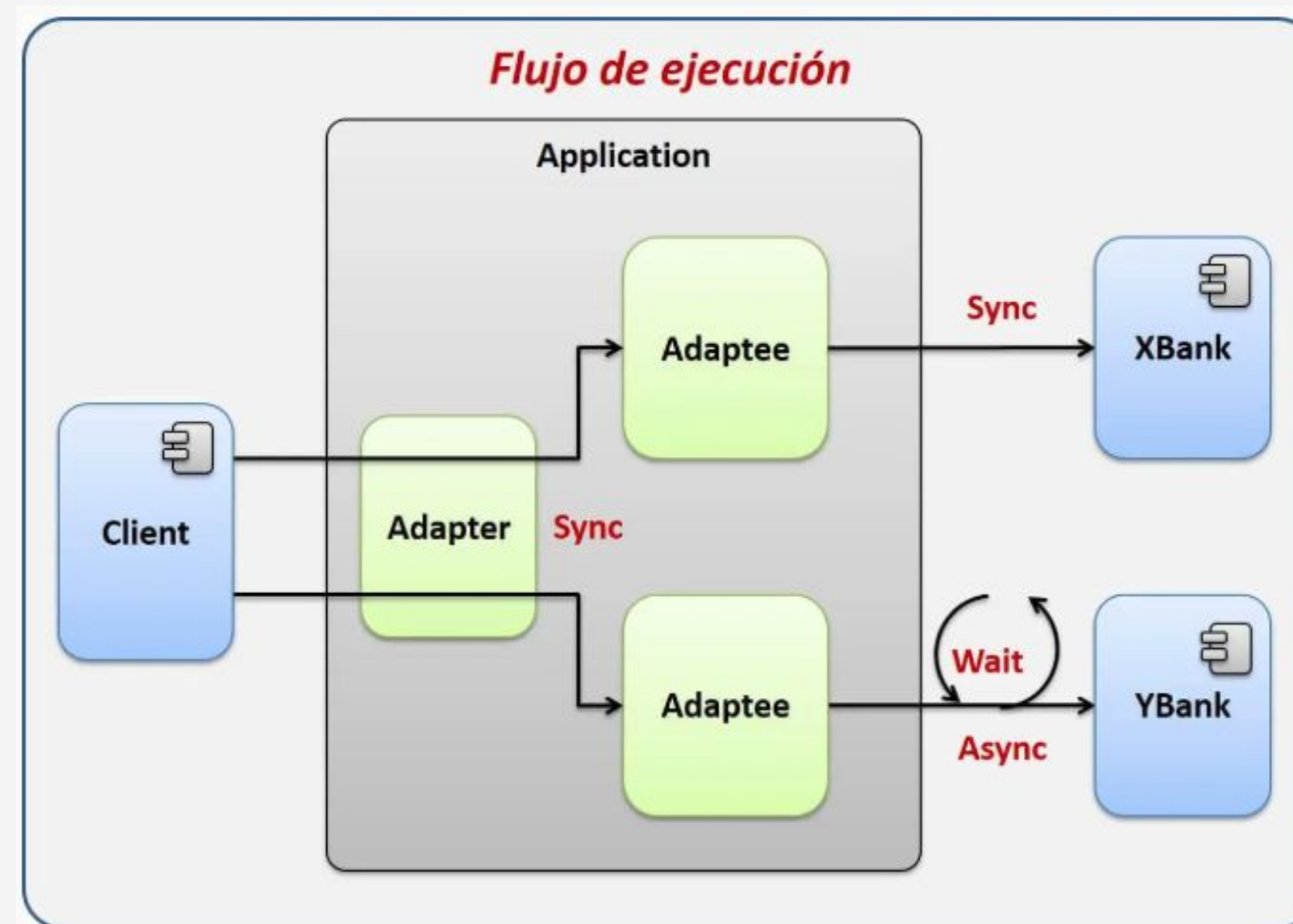
Adapter pattern – Class diagram

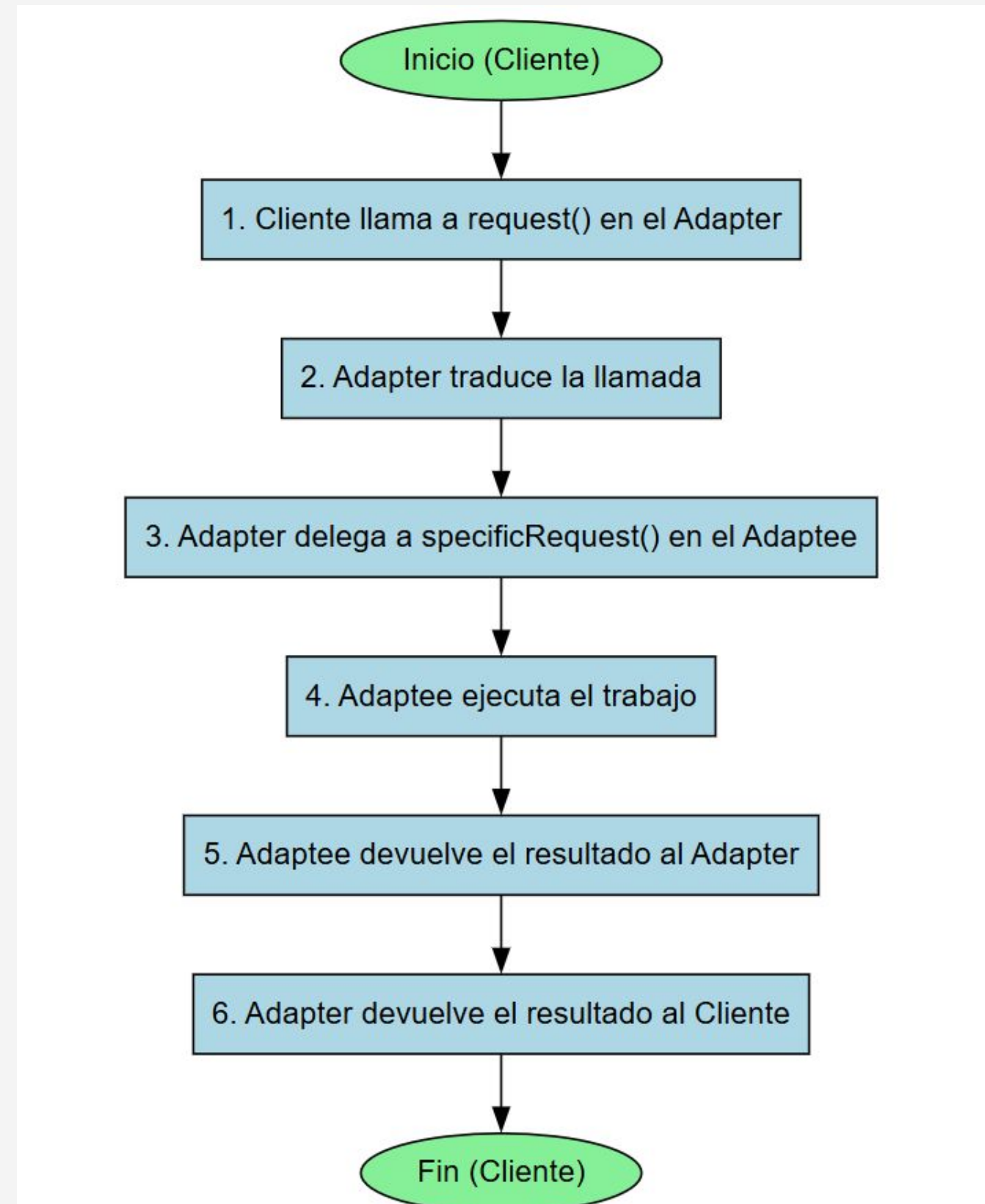


Adapter pattern – Sequence diagram



Mediante la implementación del patrón de diseño Adapter crearemos un adaptador que nos permite interactuar de forma homogénea entre dos API bancarias, las cuales nos permite aprobar créditos personales, sin embargo, las dos API proporcionadas por los bancos cuenta con interfaces diferentes y aunque su funcionamiento es prácticamente igual, las interfaces expuestas son diferentes, lo que implica tener dos implementaciones diferentes para procesar los préstamos con cada banco. Mediante este patrón crearemos un adaptador que permitirá ocultar la complejidad de cada implementación del API, exponiendo una única interface compatible con las dos API proporcionadas, además que dejáramos el camino preparado por si el día de mañana llegara una nueva API bancaria.





EJEMPLO PRÁCTICO

Tu app usa esta interfaz:

- `IPago.cobrar(monto: number): boolean`

Debes integrar una pasarela vieja que no puedes cambiar:

- `OldGateway.processPayment(value: string): "OK" | "FAIL"`

Tarea

Crea `OldGatewayAdapter` que implemente `IPago` y, dentro de `cobrar(monto)`, haga:

1. Convertir `monto (number)` a `string` con dos decimales.
2. Llamar a `OldGateway.processPayment(value)`.
3. Devolver `true` si la respuesta es `"OK"`, y `false` si es `"FAIL"`.

Regla extra

Si `monto <= 0`, devolver `false` sin llamar a la pasarela.

Criterio de aceptación

El resto del sistema sigue usando solo `IPago`. Al inyectar el adaptador, `cobrar(199.99)` termina llamando internamente `processPayment("199.99")` y retorna `true`.



PROXY

PROXY

El patrón Proxy es un patrón estructural que actúa como un sustituto o marcador de posición para otro objeto, permitiendo controlar el acceso a este. En lugar de interactuar directamente con el objeto real, el cliente interactúa con el proxy, que a su vez delega la petición al objeto real en el momento y la forma correctos.

Estructura del Patrón

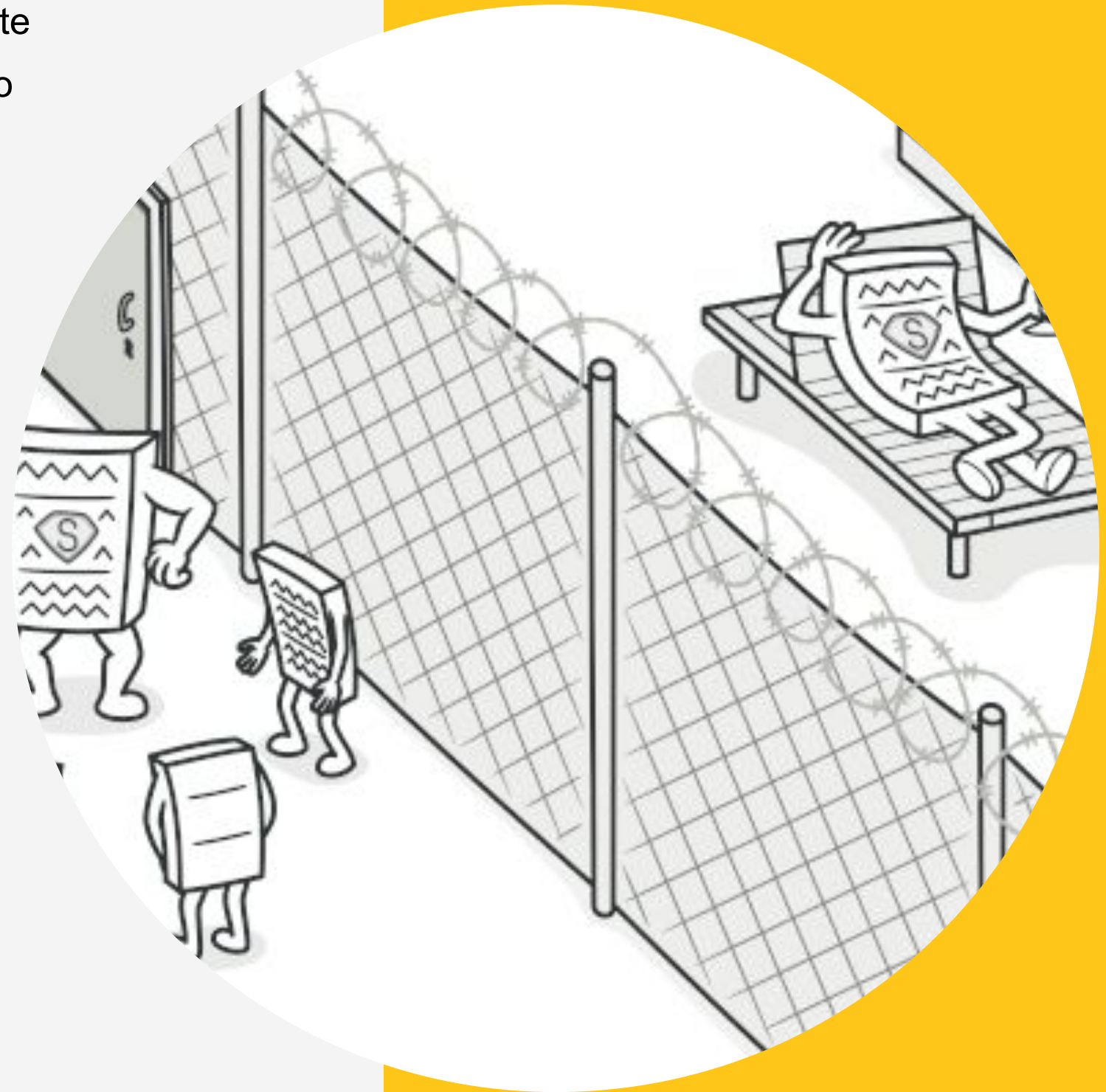
La estructura del proxy generalmente involucra cuatro componentes clave:

Subject (Sujeto): La interfaz común que tanto el **RealSubject** como el **Proxy** implementan. Esto garantiza que el **Proxy** sea intercambiable por el **RealSubject**.

- **Proxy (Sustituto):** El objeto que actúa como sustituto. Mantiene una referencia al **RealSubject** y controla el acceso a él. Puede añadir lógica adicional antes o después de delegar la petición al objeto real.

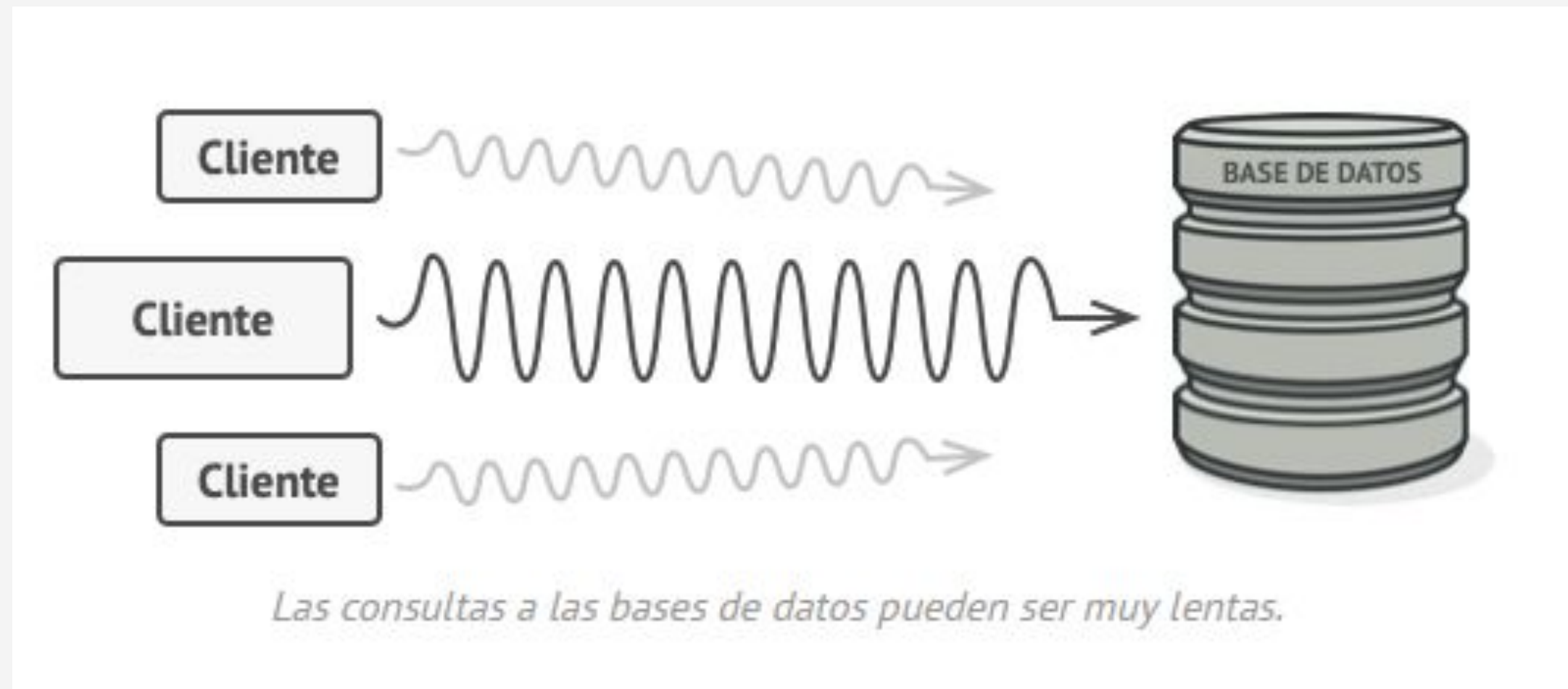
RealSubject (Sujeto Real): El objeto pesado o complejo que el **Proxy** representa. Este objeto realiza el trabajo real.

Client (Cliente): El objeto que interactúa con el **Proxy** pensando que es el **RealSubject**. El cliente no necesita saber que está usando un proxy.



PROBLEMA

¿Por qué querrías controlar el acceso a un objeto? Imagina que tienes un objeto enorme que consume una gran cantidad de recursos del sistema. Lo necesitas de vez en cuando, pero no siempre.

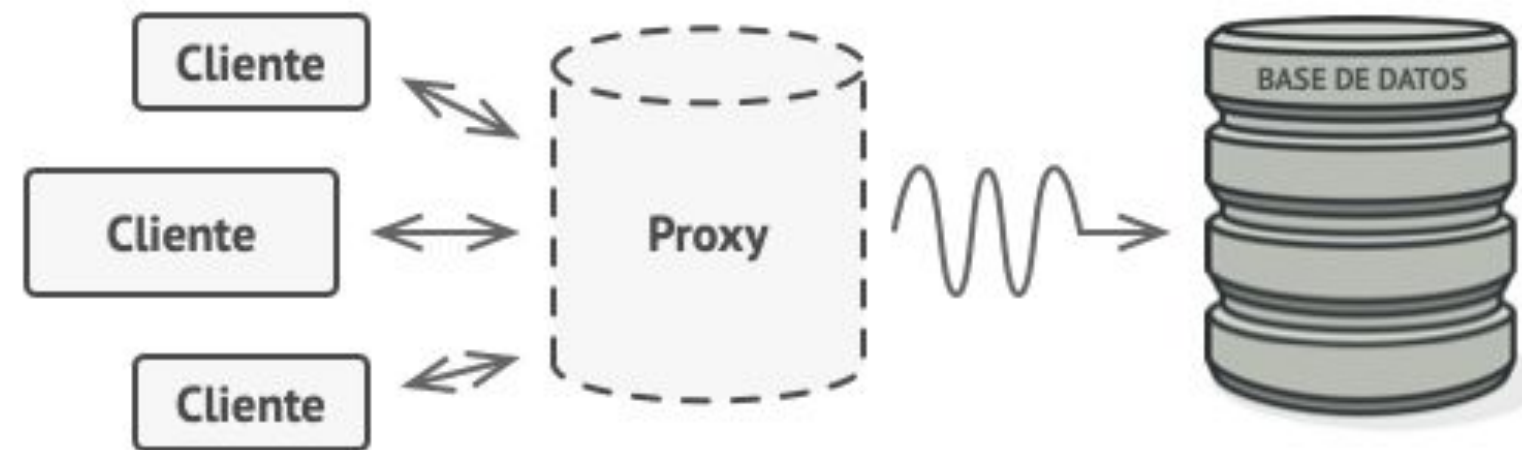


Puedes llevar a cabo una implementación diferida, es decir, crear este objeto sólo cuando sea realmente necesario. Todos los clientes del objeto tendrán que ejecutar algún código de inicialización diferida. Lamentablemente, esto seguramente generará una gran cantidad de código duplicado.

En un mundo ideal, querríamos meter este código directamente dentro de la clase de nuestro objeto, pero eso no siempre es posible. Por ejemplo, la clase puede ser parte de una biblioteca cerrada de un tercero.

SOLUCIÓN...

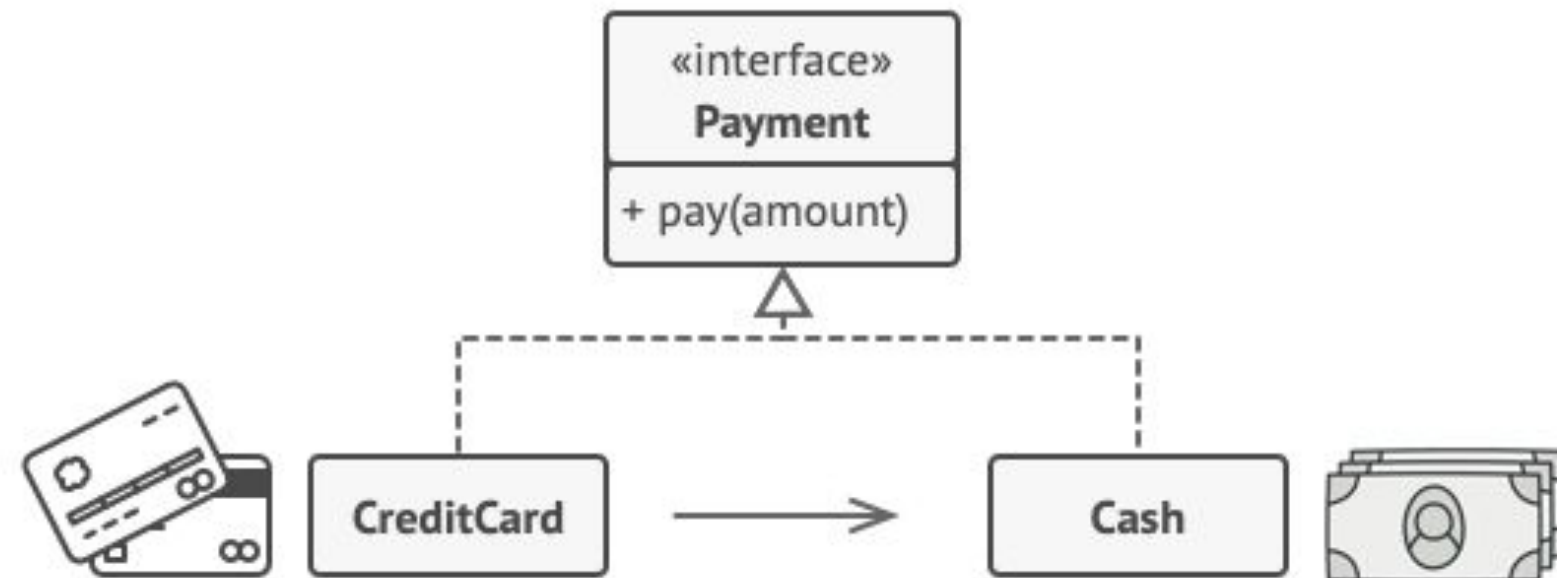
El patrón Proxy sugiere que crees una nueva clase proxy con la misma interfaz que un objeto de servicio original. Después actualizas tu aplicación para que pase el objeto proxy a todos los clientes del objeto original. Al recibir una solicitud de un cliente, el proxy crea un objeto de servicio real y le delega todo el trabajo.



El proxy se camufla como objeto de la base de datos. Puede gestionar la inicialización diferida y el caché de resultados sin que el cliente o el objeto real de la base de datos lo sepan.

Pero, ¿cuál es la ventaja? Si necesitas ejecutar algo antes o después de la lógica primaria de la clase, el proxy te permite hacerlo sin cambiar esa clase. Ya que el proxy implementa la misma interfaz que la clase original, puede pasarse a cualquier cliente que espere un objeto de servicio real.

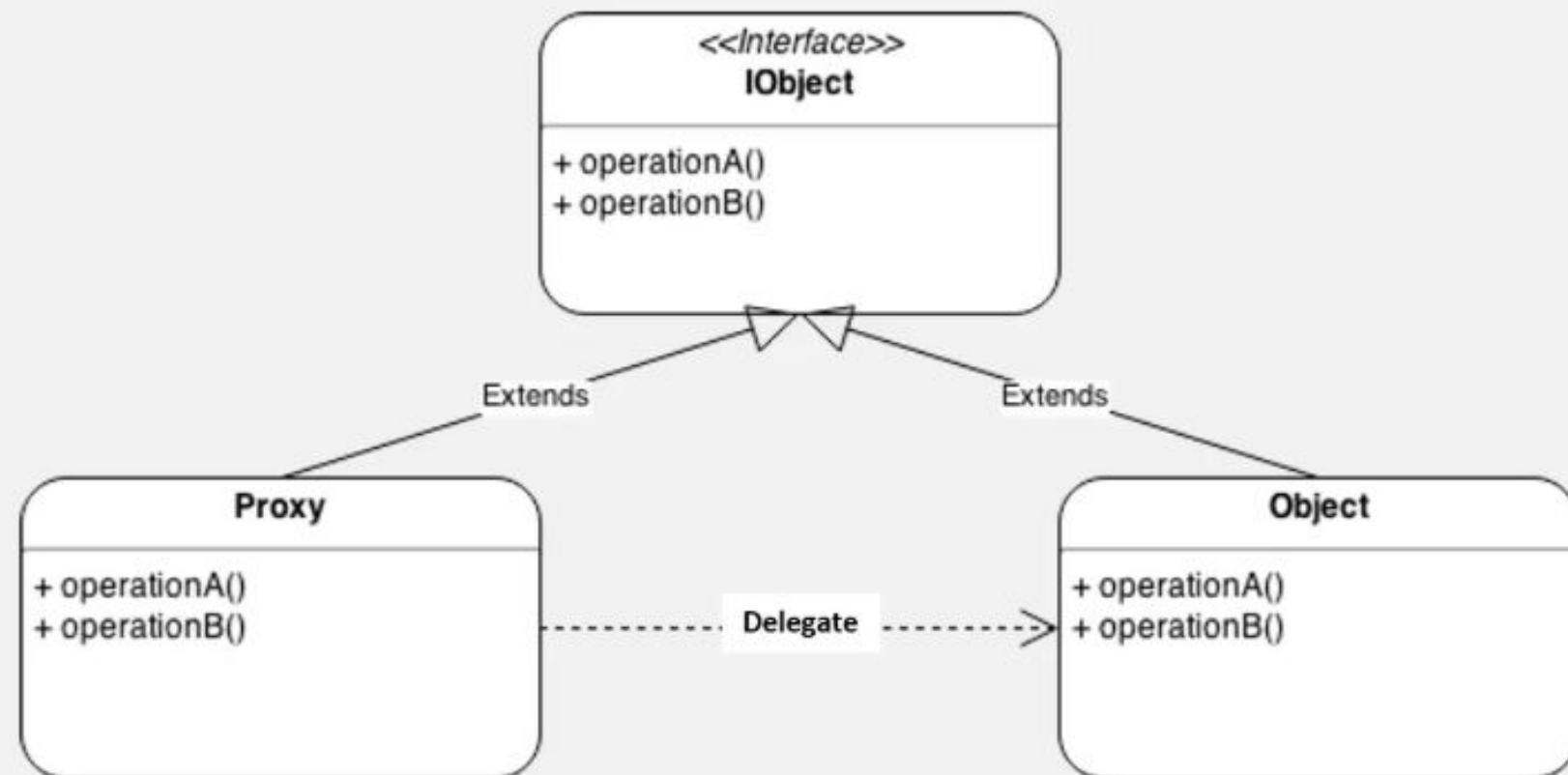
ANALOGÍA DEL MUNDO REAL



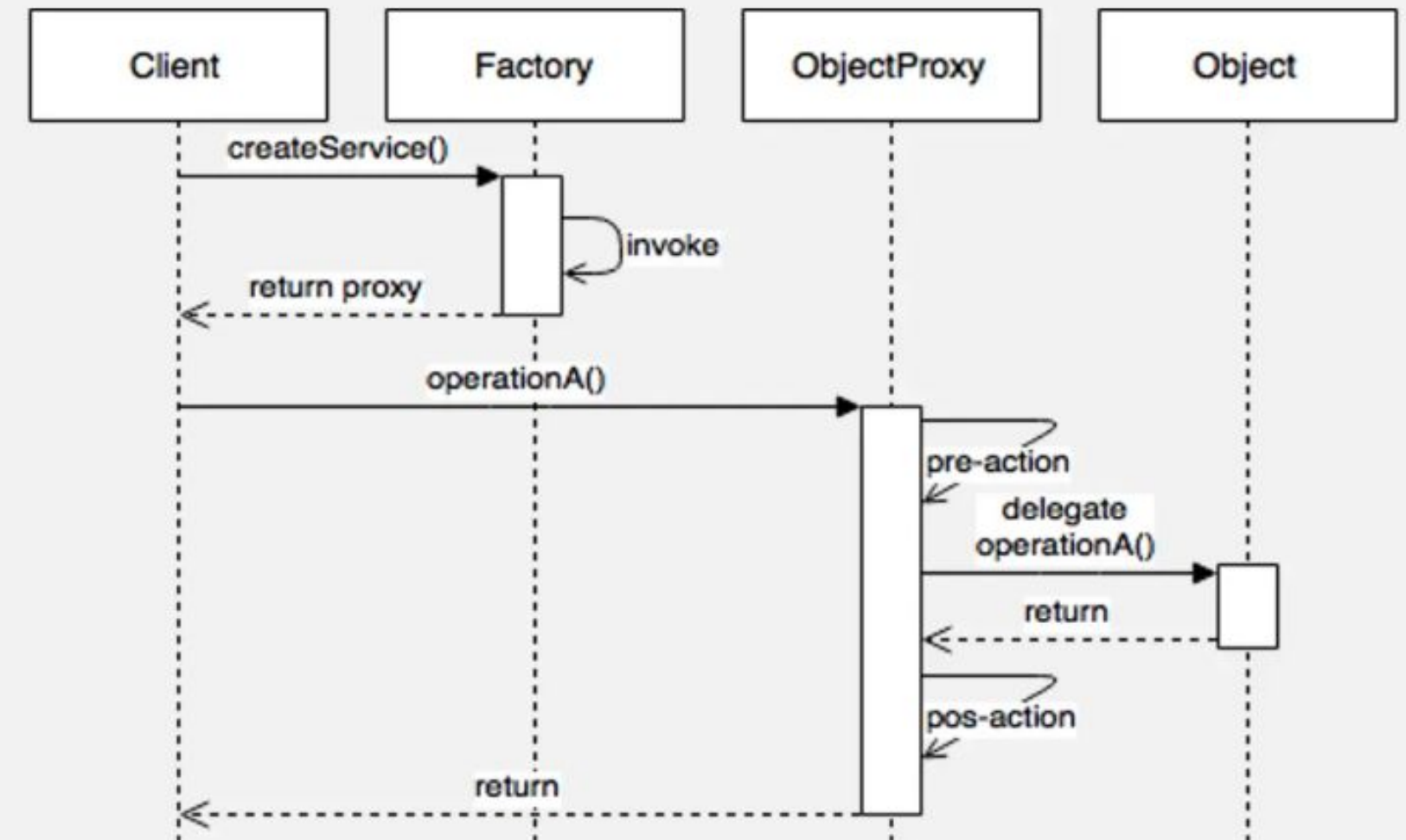
Las tarjetas de crédito pueden utilizarse para realizar pagos tanto como el efectivo.

Una tarjeta de crédito es un proxy de una cuenta bancaria, que, a su vez, es un proxy de un manojo de billetes. Ambos implementan la misma interfaz, por lo que pueden utilizarse para realizar un pago. El consumidor se siente genial porque no necesita llevar un montón de efectivo encima. El dueño de la tienda también está contento porque los ingresos de la transacción se añaden electrónicamente a la cuenta bancaria de la tienda sin el riesgo de perder el depósito o sufrir un robo de camino al banco.

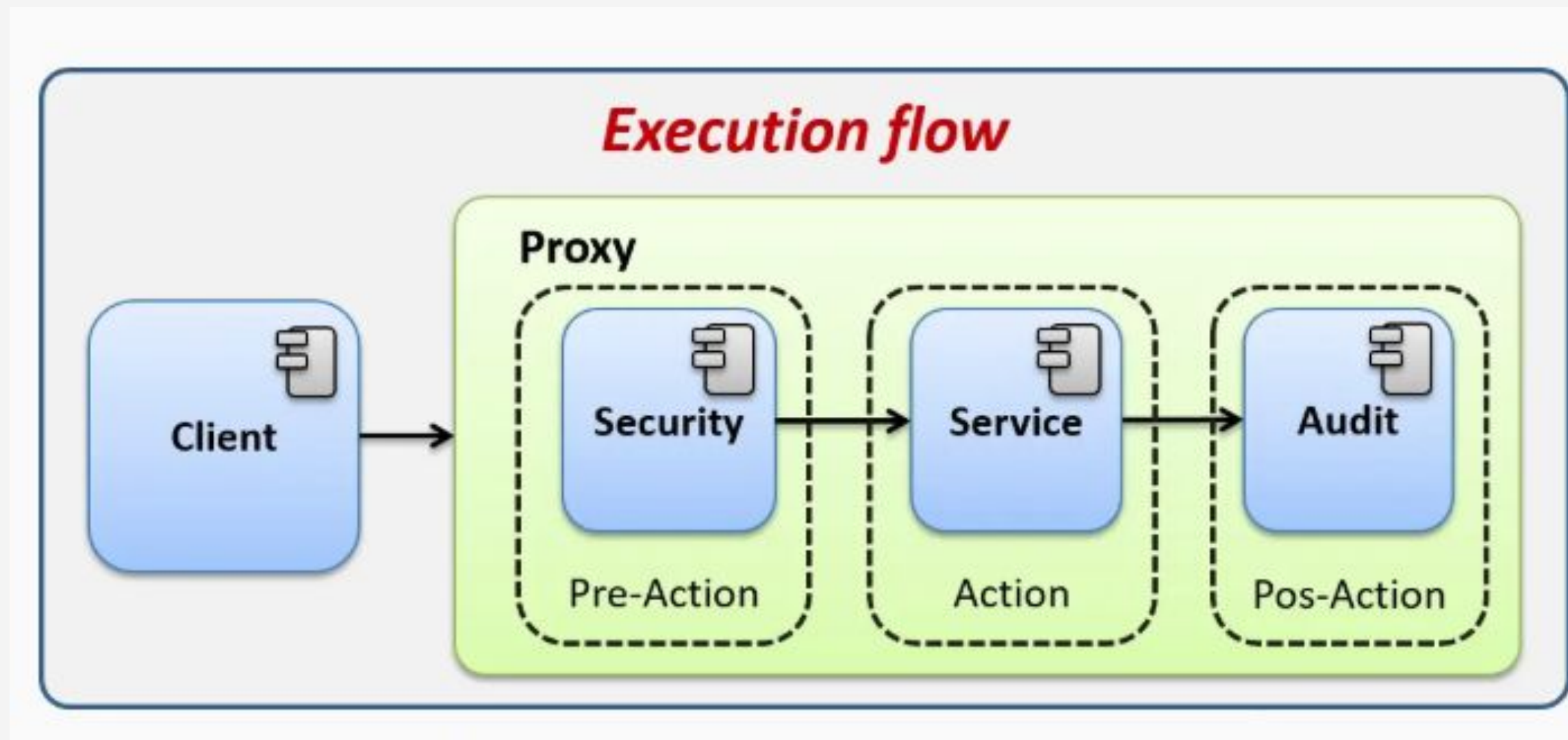
Proxy pattern – Class diagram

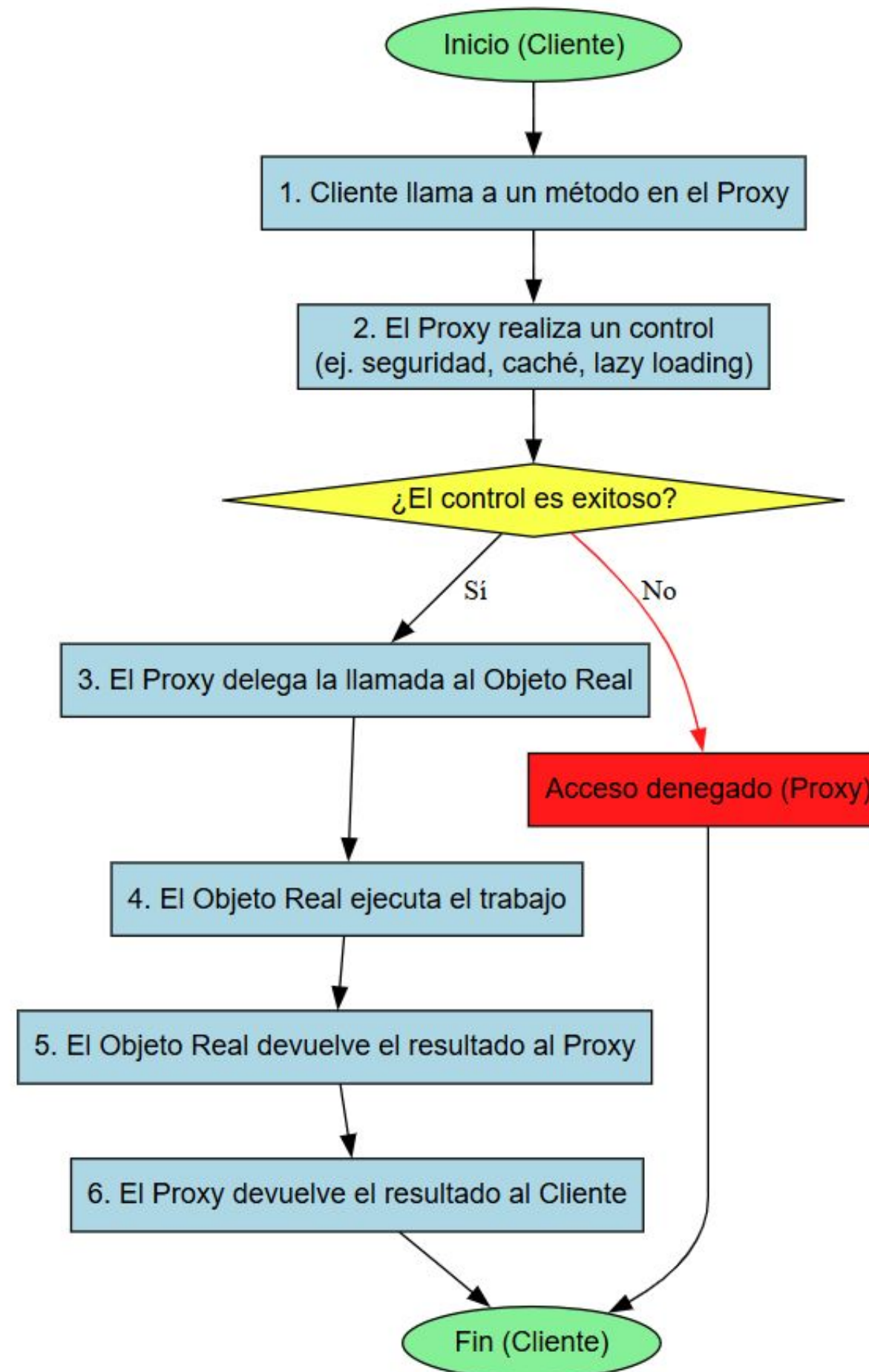


Proxy pattern – Diagram of sequence



Mediante la implementación del patrón de diseño Proxy crearemos un mecanismo de seguridad, el cual intercepte las ejecuciones de procesos para validar si el usuario que intenta ejecutar cuenta con los privilegios necesarios, evitando que usuarios no autorizados los ejecuten, además, una vez que el proceso es ejecutado, se auditará la ejecución y quedará un registro de la ejecución. Todo esto se realizará sin que el usuario se dé cuenta, pues el proxy envolverá la lógica de seguridad.





EJEMPLO PRACTICO

Enunciado: Quieres crear un servicio de visualización de imágenes para una galería en línea. Las imágenes son de muy alta resolución y se cargan lentamente desde un servidor. Diseña un sistema donde el cliente interactúa con un ImageProxy en lugar de con el RealImage. El Proxy debe tener las siguientes funcionalidades:

- Muestra un marcador de posición (un placeholder o un mensaje de carga) de forma inmediata.

- Solo carga la imagen real desde el servidor cuando el usuario intenta verla (carga perezosa o lazy loading).

- Almacena en caché la imagen cargada para no tener que volver a cargarla si el usuario la solicita de nuevo.



FACADE



FACADE

El patrón de diseño Facade es un patrón estructural que ofrece una interfaz simple y unificada a un subsistema complejo. Su propósito es simplificar la interacción del cliente con un conjunto de clases, ocultando su complejidad y exponiendo un único punto de entrada.

Estructura del Patrón

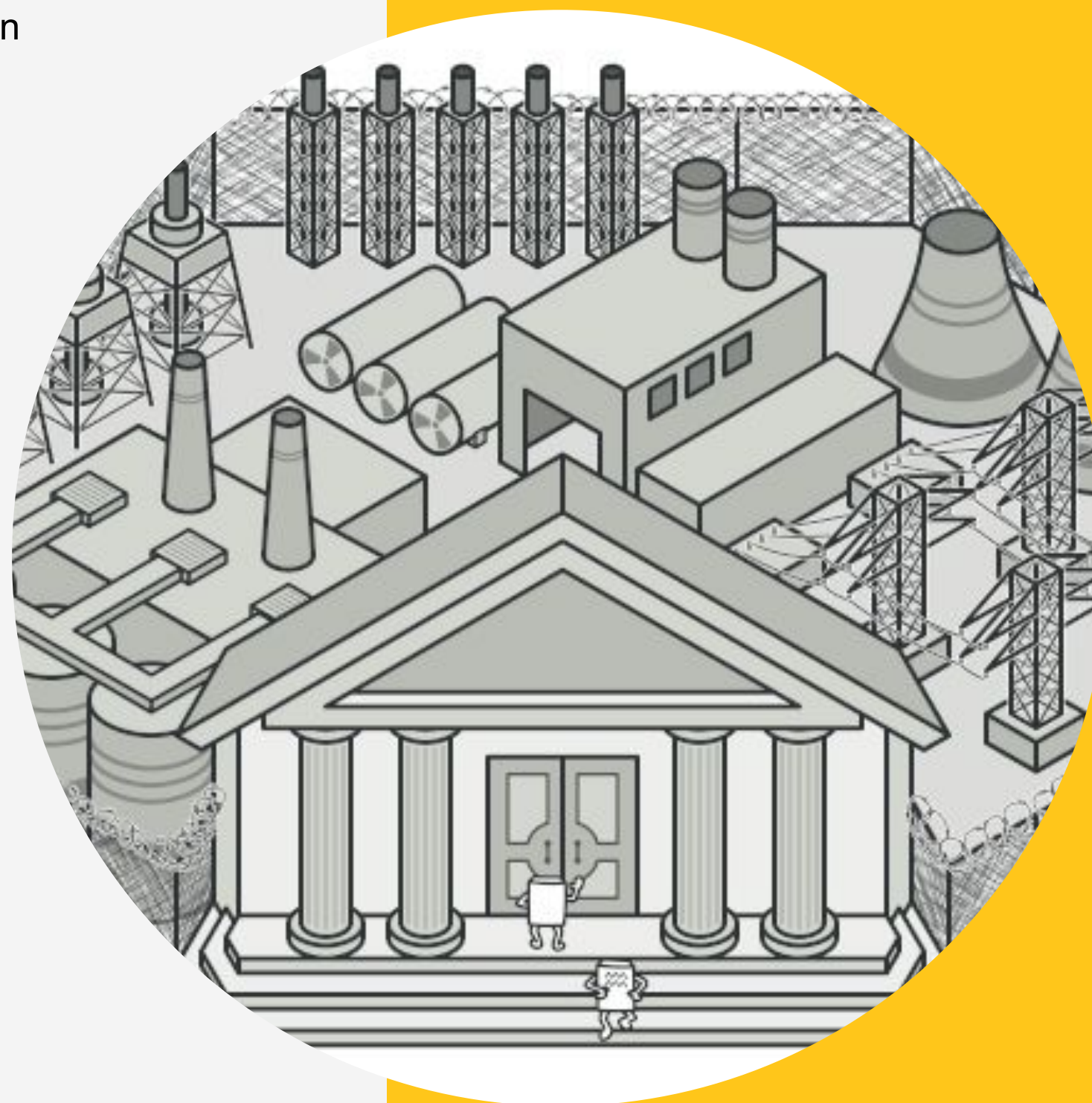
La estructura del Facade generalmente involucra cuatro componentes clave:

Facade (Fachada): La clase principal que proporciona la interfaz unificada y simplificada.

Internamente, mantiene referencias a las clases del subsistema y delega las peticiones del cliente a los objetos apropiados

- **Client (Cliente):** El objeto que usa el Facade para interactuar con el subsistema. El cliente no conoce la complejidad interna del subsistema y solo usa la interfaz simplificada que ofrece la Facade.

Subsystem Classes (Clases del Subsistema): Las clases complejas y de bajo nivel que realizan el trabajo real. El cliente no interactúa directamente con ellas; en su lugar, se comunican a través de la Facade.



PROBLEMA

Imagina que debes lograr que tu código trabaje con un amplio grupo de objetos que pertenecen a una sofisticada biblioteca o framework. Normalmente, debes inicializar todos esos objetos, llevar un registro de las dependencias, ejecutar los métodos en el orden correcto y así sucesivamente.

Como resultado, la lógica de negocio de tus clases se vería estrechamente acoplada a los detalles de implementación de las clases de terceros, haciéndola difícil de comprender y mantener.

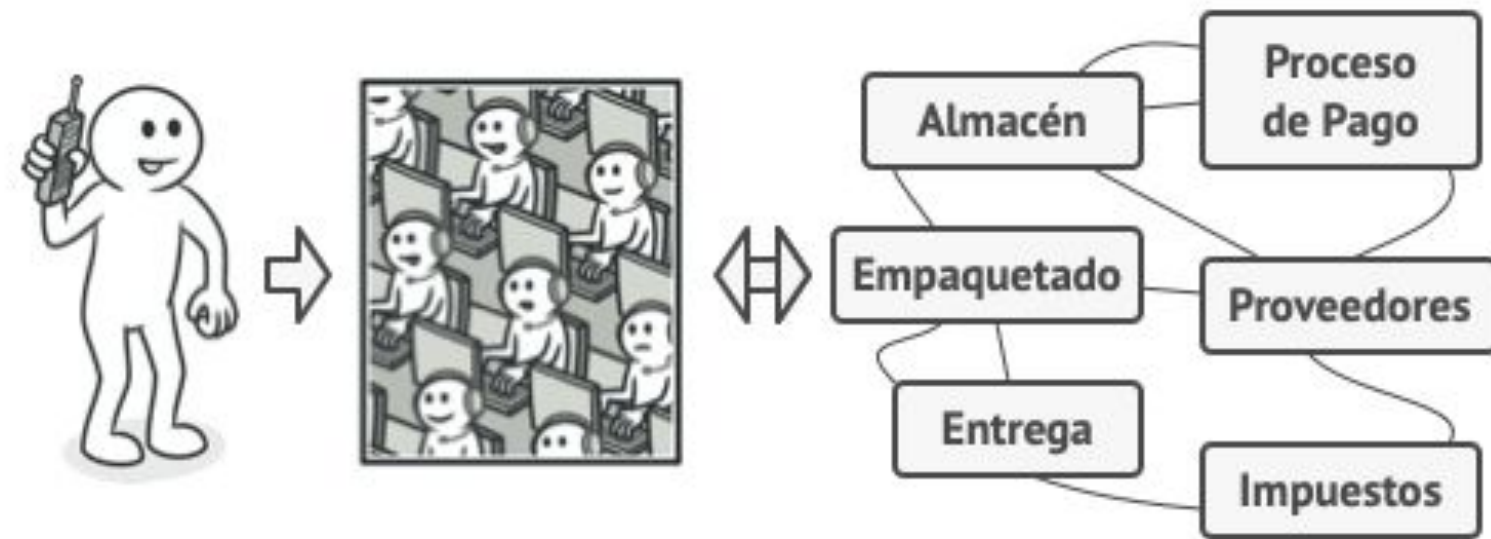
SOLUCIÓN

Una fachada es una clase que proporciona una interfaz simple a un subsistema complejo que contiene muchas partes móviles. Una fachada puede proporcionar una funcionalidad limitada en comparación con trabajar directamente con el subsistema. Sin embargo, tan solo incluye las funciones realmente importantes para los clientes.

Tener una fachada resulta útil cuando tienes que integrar tu aplicación con una biblioteca sofisticada con decenas de funciones, de la cual sólo necesitas una pequeña parte.

Por ejemplo, una aplicación que sube breves vídeos divertidos de gatos a las redes sociales, podría potencialmente utilizar una biblioteca de conversión de vídeo profesional. Sin embargo, lo único que necesita en realidad es una clase con el método simple `codificar(nombreDelArchivo, formato)`. Una vez que crees dicha clase y la conectes con la biblioteca de conversión de vídeo, tendrás tu primera fachada.

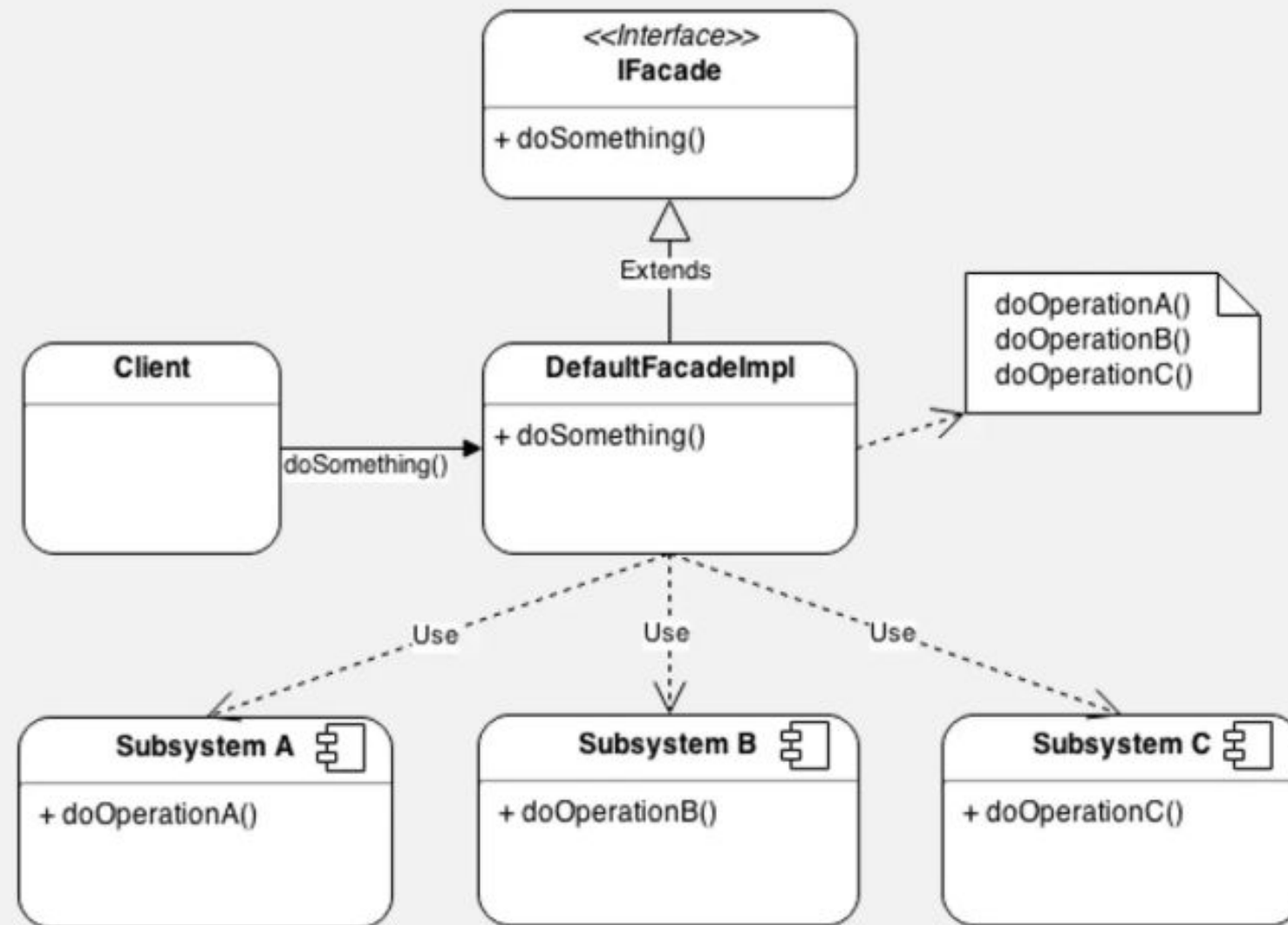
ANALOGÍA DEL MUNDO REAL



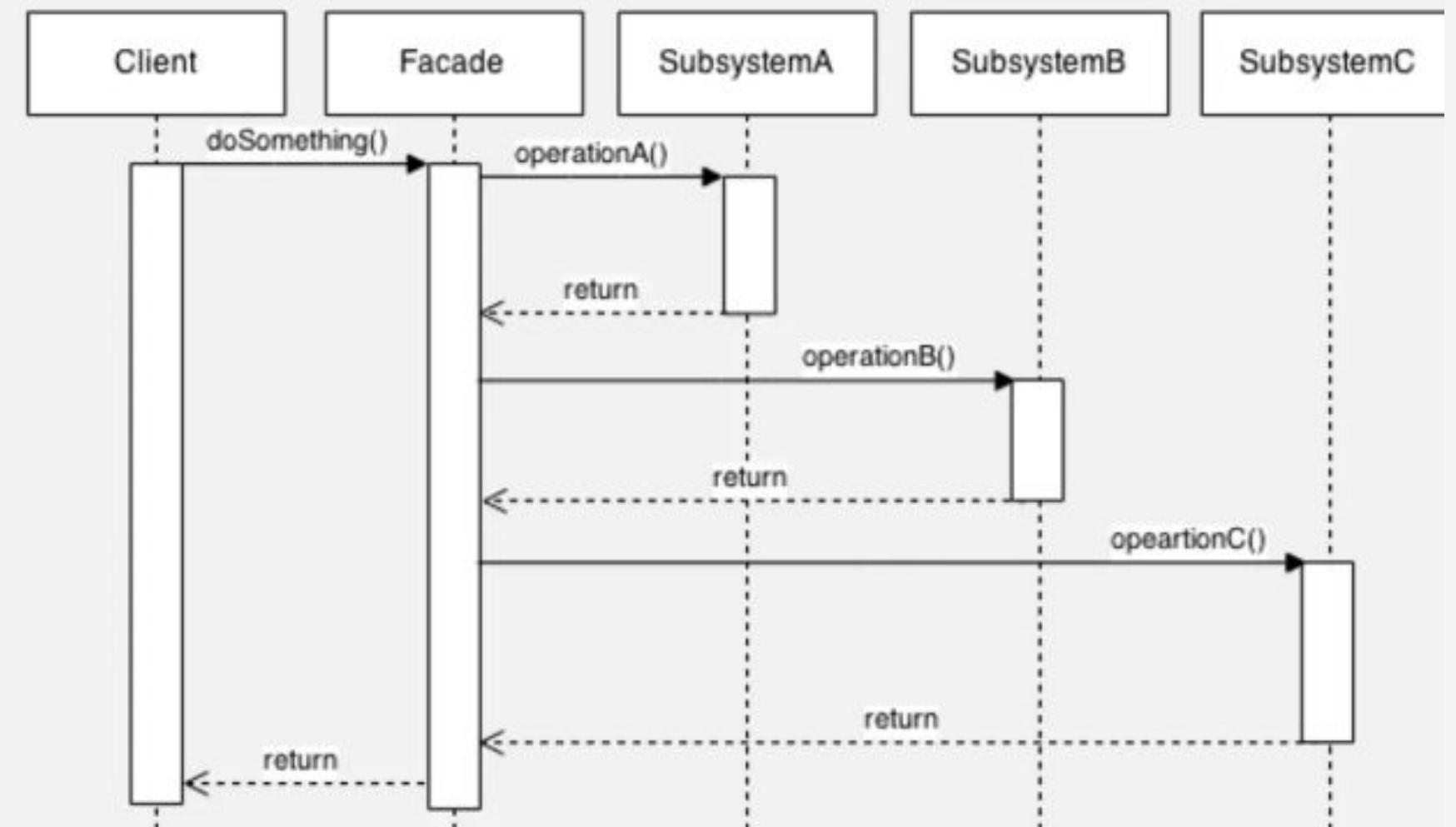
Haciendo pedidos por teléfono.

Cuando llamas a una tienda para hacer un pedido por teléfono, un operador es tu fachada a todos los servicios y departamentos de la tienda. El operador te proporciona una sencilla interfaz de voz al sistema de pedidos, pasarelas de pago y varios servicios de entrega.

Facade pattern – Class diagram

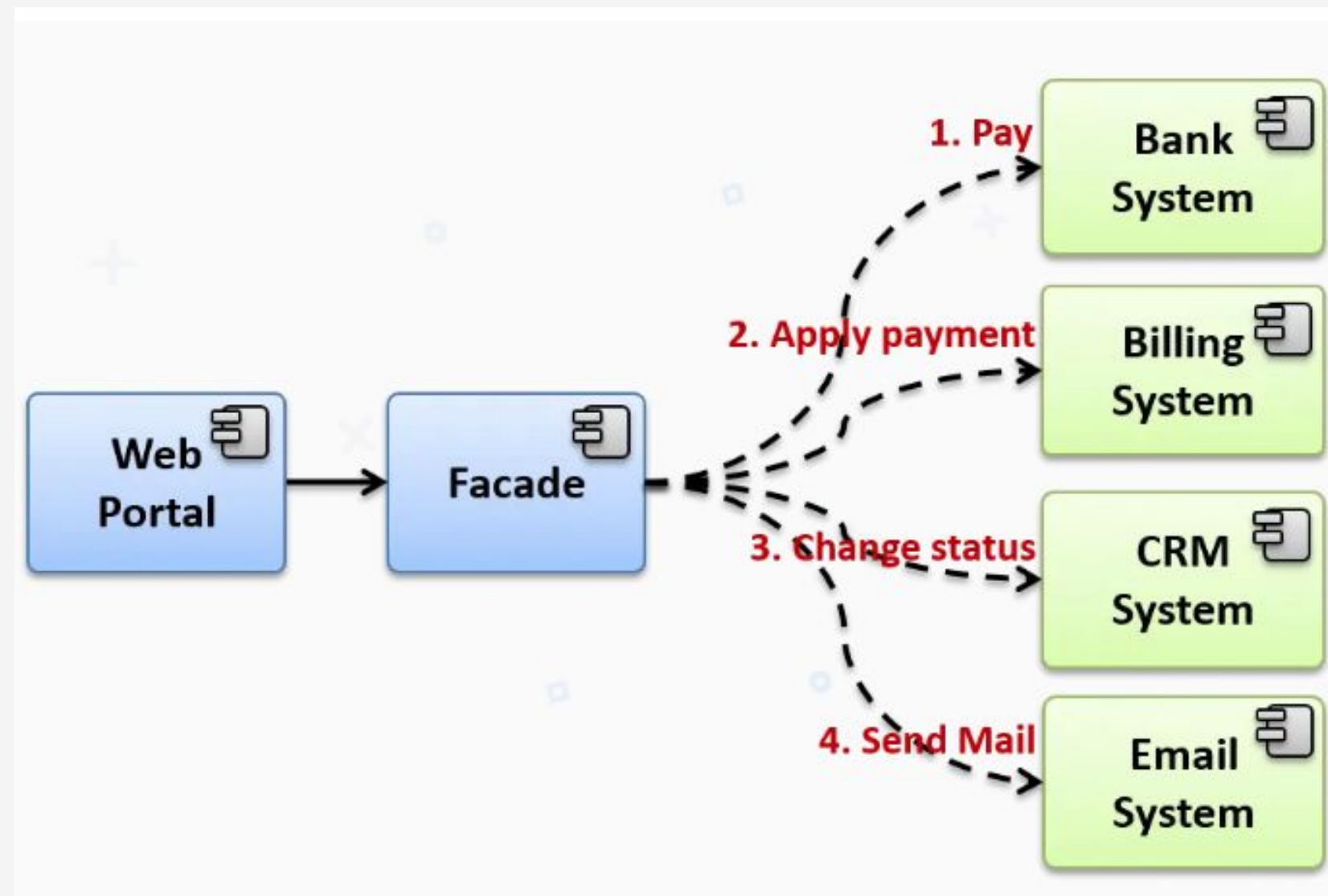


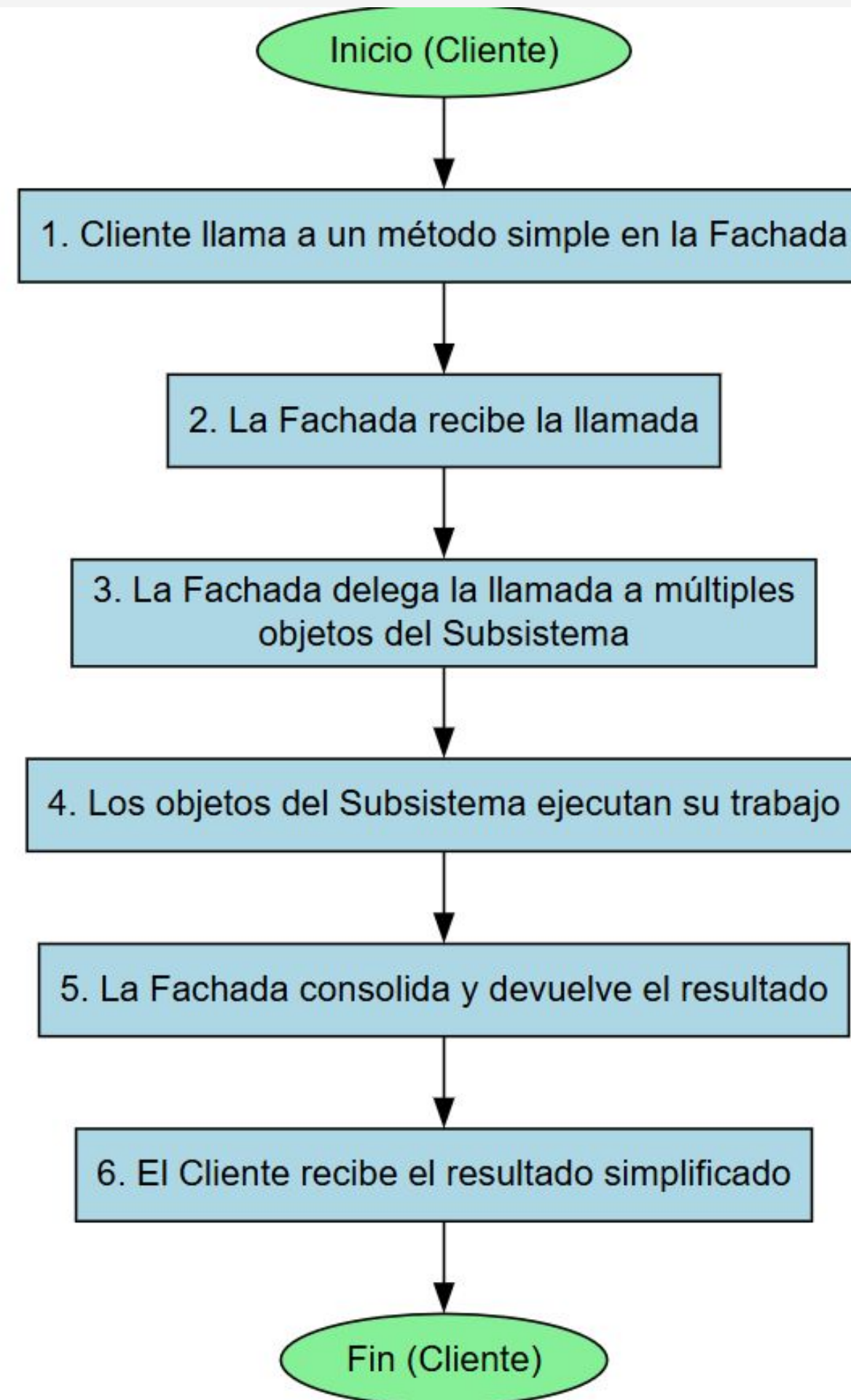
Facade pattern – Diagram of sequence



CASO REAL

Mediante la implementación del patrón de diseño Facade implementaremos un sistema que permite realizar pagos en línea, para lo cual será necesario interactuar con varios sistemas, dichos sistemas conllevan una cierta complejidad, por lo que interactuar con todos los subsistemas puede ser muy complicado, sobre todo para los programadores que no tienen contexto del funcionamiento de los subsistemas. Por lo cual se implementará una fachada que permita exponer operaciones de alto nivel, las cuales se encarguen de interactuar con los subsistemas y abstraer a los programadores de la complejidad de interactuar con dichos sistemas.





EJEMPLO PRÁCTICO

La app necesita **registrar** y **loguear** usuarios. Internamente hay varios pasos (validar email, “hashear” password, guardar usuario, emitir token de sesión).

Crea una **Fachada AuthFacade** con métodos:

- `register(email, password): Promise<{ id: number }>`
- `login(email, password): Promise<{ token: string }>`
- `getProfile(token): Promise<{ id: number; email: string }>`
- `logout(token): Promise<void>`

Los clientes solo llaman a la fachada; los subsistemas quedan ocultos.



DECORATOR



DECORATOR

El patrón de diseño Decorator (Decorador) es un patrón estructural que permite añadir nuevas funcionalidades a un objeto de forma dinámica sin modificar su código fuente original.

Esto se logra envolviendo el objeto original en un "decorador" que implementa la misma interfaz, de manera que el cliente no se dé cuenta de que está tratando con un objeto **Estructura del Patrón** mejorado.

La estructura del Decorator generalmente involucra cuatro componentes clave:

Component (Componente):
La interfaz común que tanto el objeto base como los decoradores implementan. Esto garantiza que todos los objetos pueden ser tratados de la misma manera por el cliente.

- **Decorator (Decorador):** Una clase abstracta que implementa la interfaz Component. Contiene una referencia a un objeto Component, lo que le permite envolverlo.

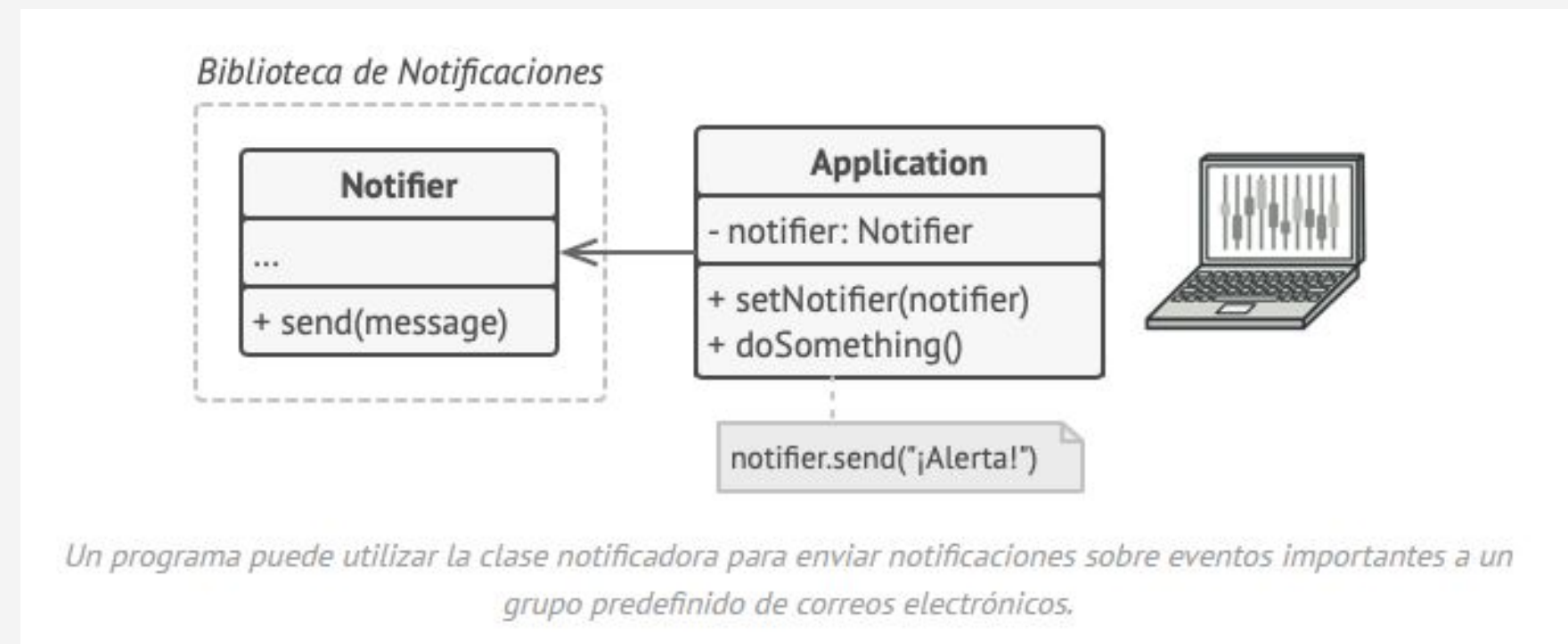
ConcreteComponent (Componente Concreto):
La clase original que recibe las nuevas funcionalidades. Este es el objeto que queremos "decorar".

ConcreteDecorator (Decorador Concreto): Las clases que añaden funcionalidades específicas. Cada una de estas clases implementa la lógica de la decoración y delega la llamada al objeto envuelto para asegurarse de que el comportamiento original se



PROBLEMA

Imagina que estás trabajando en una biblioteca de notificaciones que permite a otros programas notificar a sus usuarios acerca de eventos importantes. La versión inicial de la biblioteca se basaba en la clase Notificador que solo contaba con unos cuantos campos, un constructor y un único método send. El método podía aceptar un argumento de mensaje de un cliente y enviar el mensaje a una lista de correos electrónicos que se pasaban a la clase notificadora a través de su constructor. Una aplicación de un tercero que actuaba como cliente debía crear y configurar el objeto notificador una vez y después utilizarlo cada vez que sucediera algo importante.



En cierto momento te das cuenta de que los usuarios de la biblioteca esperan algo más que unas simples notificaciones por correo. A muchos de ellos les gustaría recibir mensajes SMS sobre asuntos importantes. Otros querrían recibir las notificaciones por Facebook y, por supuesto, a los usuarios corporativos les encantaría recibir notificaciones por Slack.

En cierto momento te das cuenta de que los usuarios de la biblioteca esperan algo más que unas simples notificaciones por correo. A muchos de ellos les gustaría recibir mensajes SMS sobre asuntos importantes. Otros querrían recibir las notificaciones por Facebook y, por supuesto, a los usuarios corporativos les encantaría recibir notificaciones por Slack.



No puede ser muy complicado ¿verdad? Extendiste la clase Notificador y metiste los métodos adicionales de notificación dentro de nuevas subclases. Ahora el cliente debería instanciar la clase notificadora deseada y utilizarla para el resto de notificaciones.

Pero entonces alguien te hace una pregunta razonable: “¿Por qué no se pueden utilizar varios tipos de notificación al mismo tiempo? Si tu casa está en llamas, probablemente quieras que te informen a través de todos los canales”.

Intentaste solucionar ese problema creando subclases especiales que combinaban varios métodos de notificación dentro de una clase. **Sin embargo, enseguida resultó evidente que esta solución inflaría el código en gran medida, no sólo el de la biblioteca, sino también el código cliente.**

SOLUCIÓN...

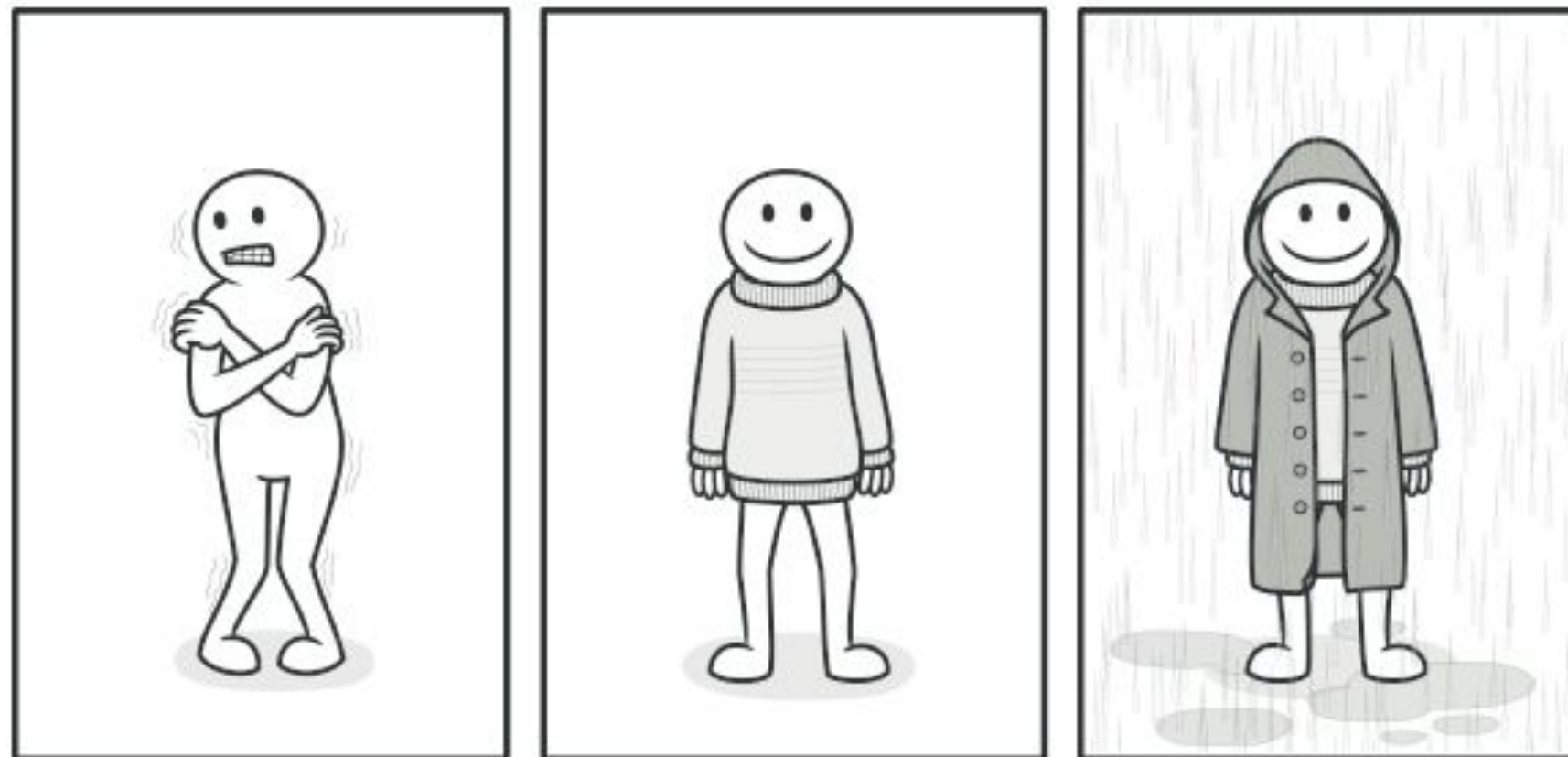
Cuando tenemos que alterar la funcionalidad de un objeto, lo primero que se viene a la mente es extender una clase. No obstante, la herencia tiene varias limitaciones importantes de las que debes ser consciente.

- La herencia es estática. No se puede alterar la funcionalidad de un objeto existente durante el tiempo de ejecución. Sólo se puede sustituir el objeto completo por otro creado a partir de una subclase diferente.
- Las subclases sólo pueden tener una clase padre. En la mayoría de lenguajes, la herencia no permite a una clase heredar comportamientos de varias clases al mismo tiempo.

Una de las formas de superar estas limitaciones es empleando la Agregación o la Composición en lugar de la Herencia. Ambas alternativas funcionan prácticamente del mismo modo: un objeto tiene una referencia a otro y le delega parte del trabajo, mientras que con la herencia, el propio objeto puede realizar ese trabajo, heredando el comportamiento de su superclase.



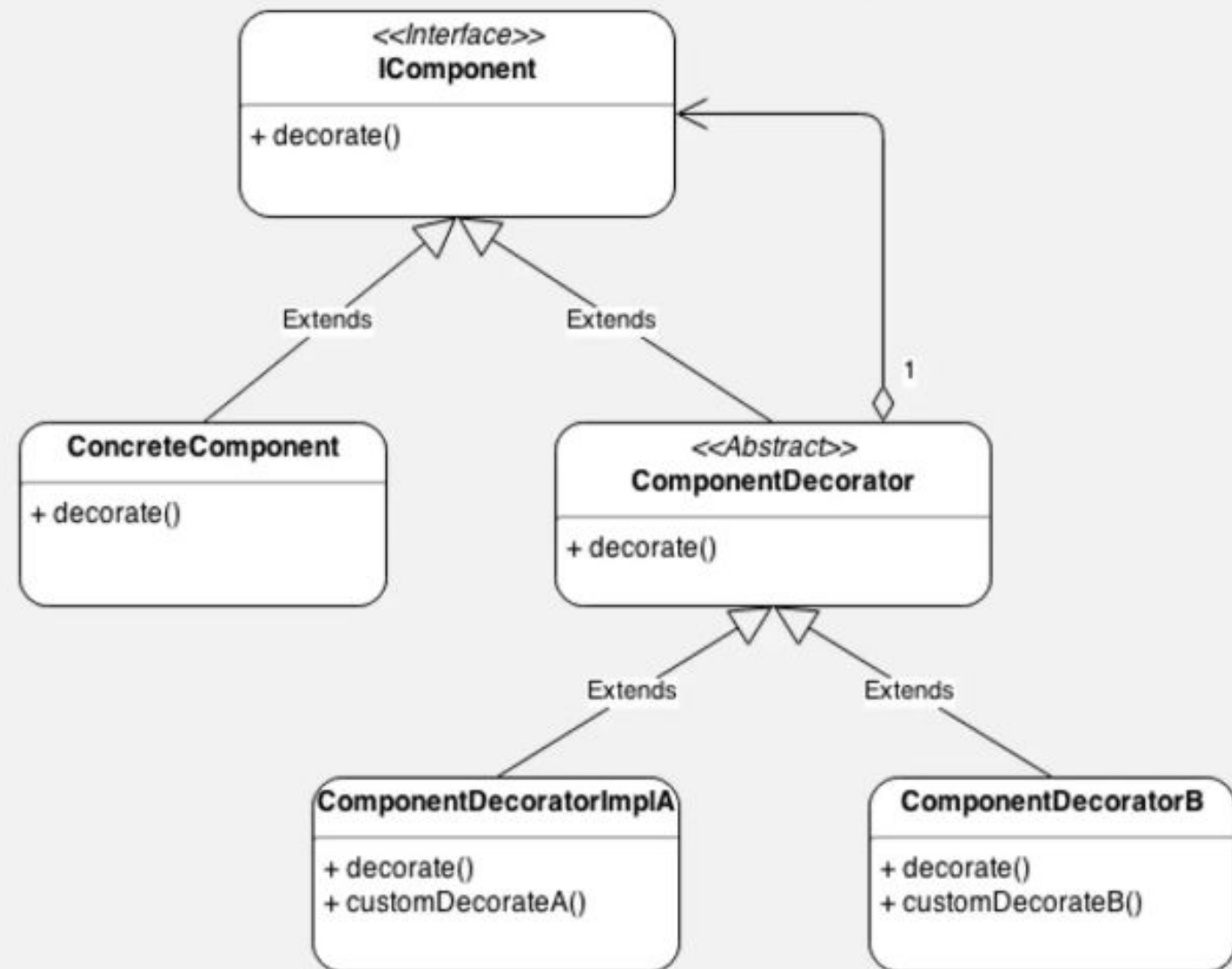
ANALOGÍA DEL MUNDO REAL



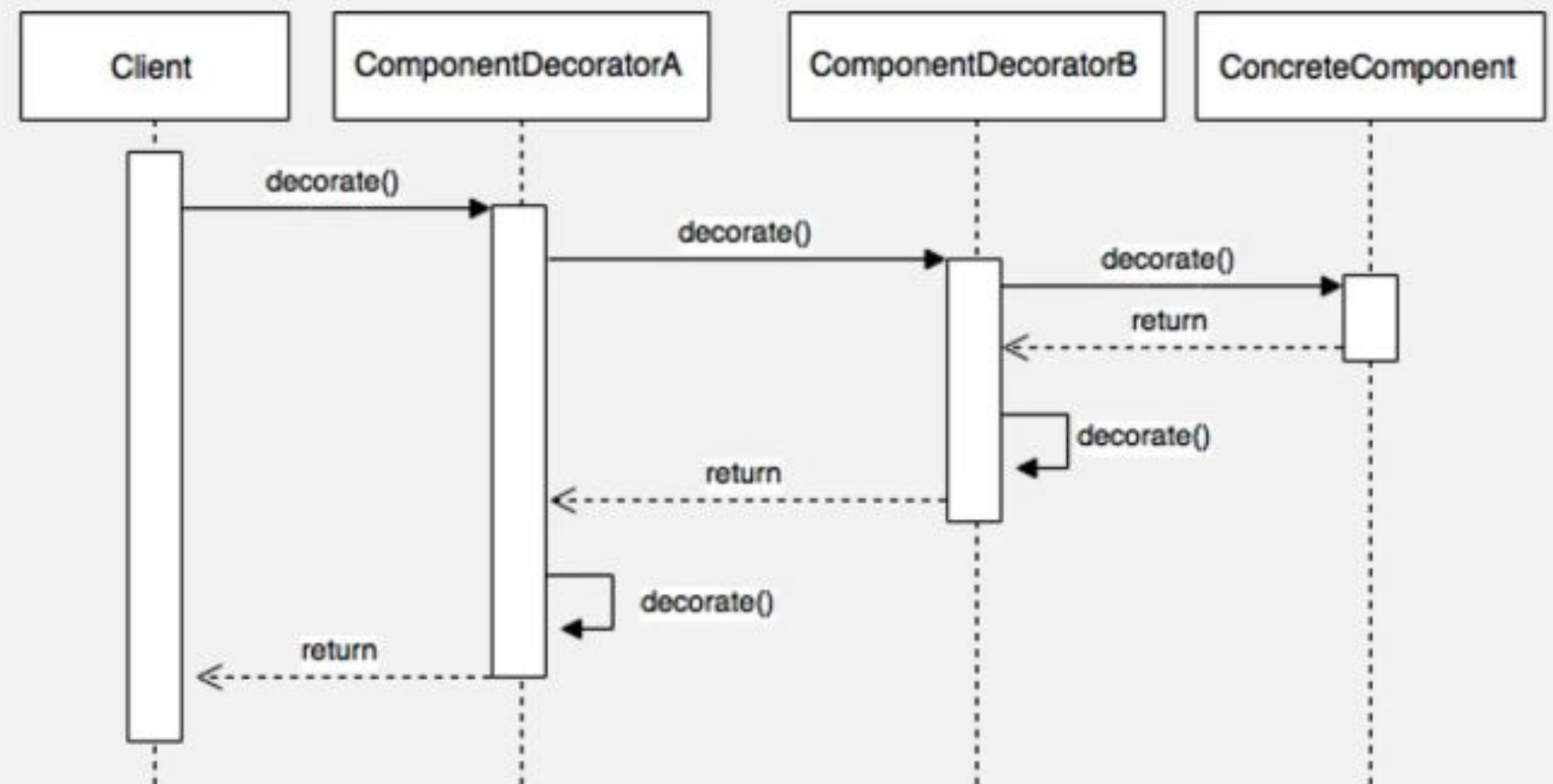
Obtienes un efecto combinado vistiendo varias prendas de ropa.

Vestir ropa es un ejemplo del uso de decoradores. Cuando tienes frío, te cubres con un suéter. Si sigues teniendo frío a pesar del suéter, puedes ponerte una chaqueta encima. Si está lloviendo, puedes ponerte un impermeable. Todas estas prendas “extienden” tu comportamiento básico pero no son parte de ti, y puedes quitarte fácilmente cualquier prenda cuando lo desees.

Decorator pattarn – Class diagram

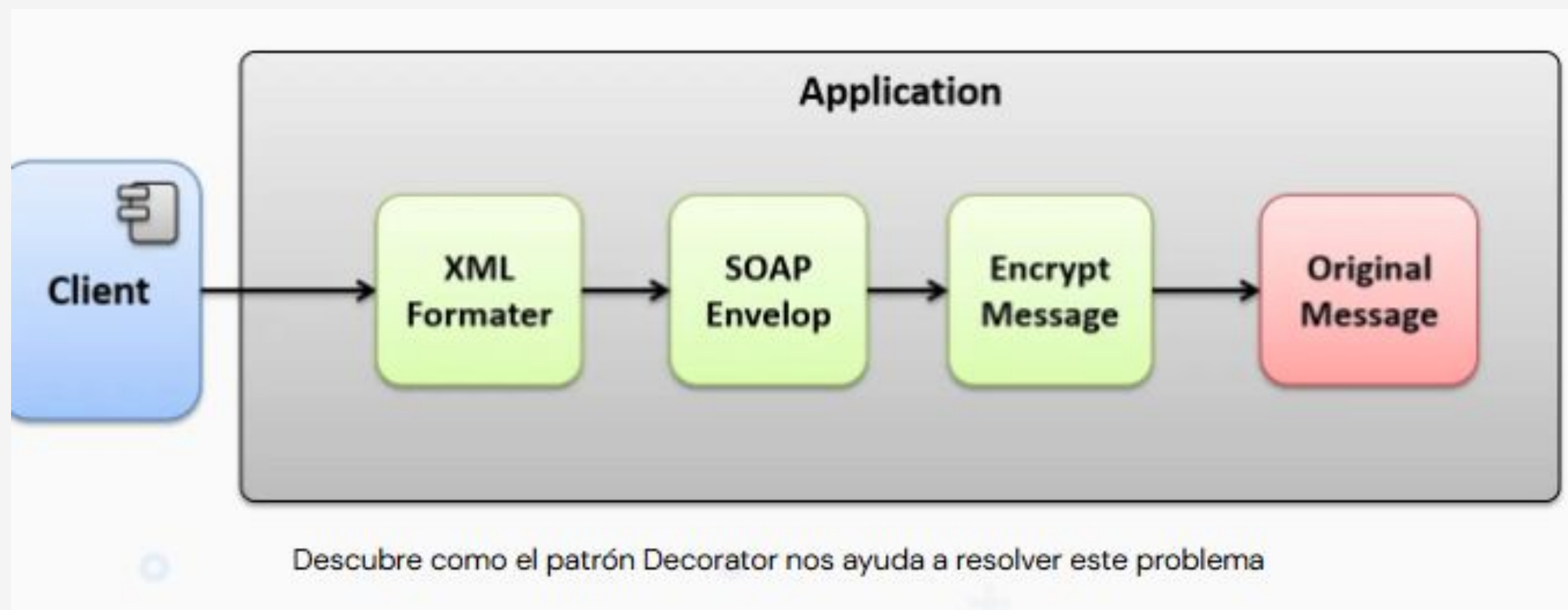


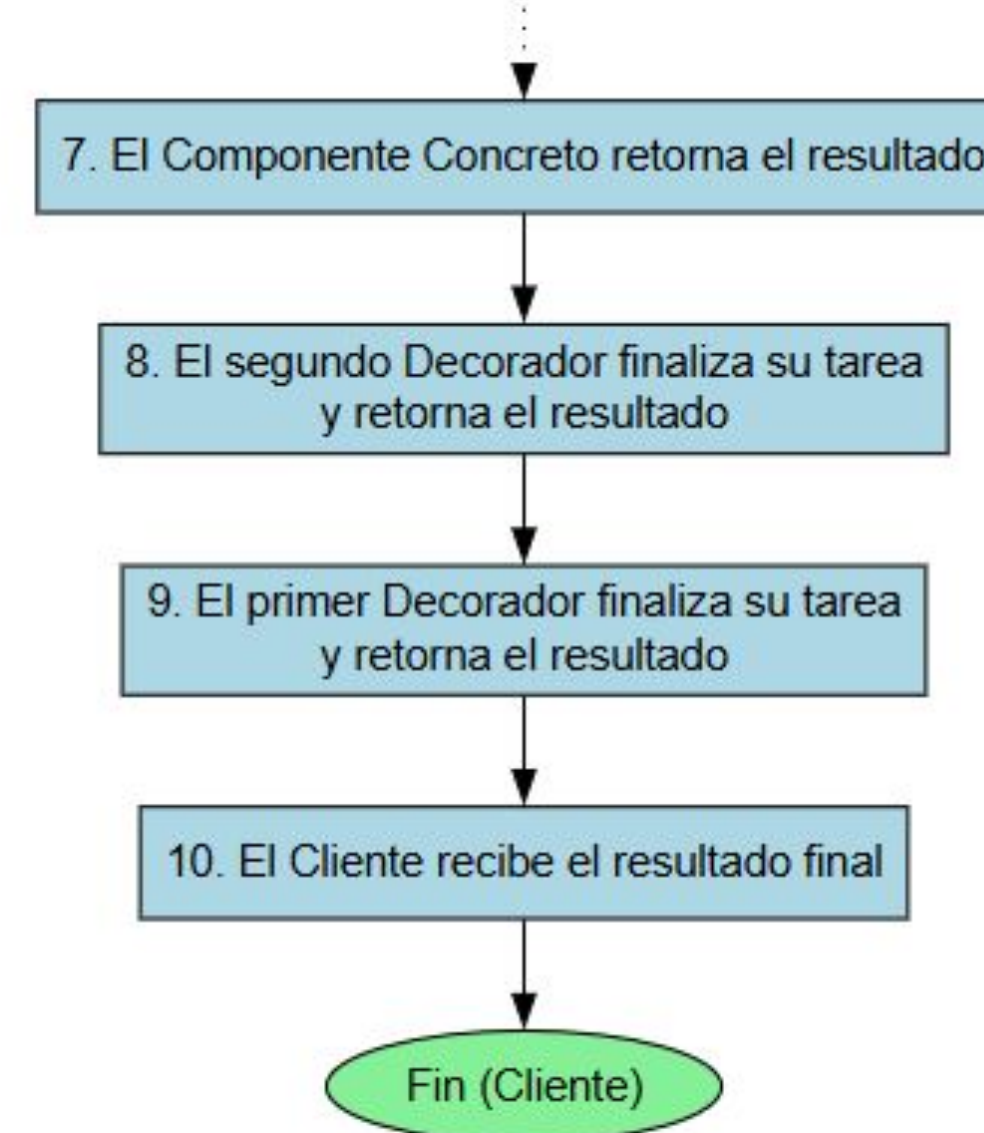
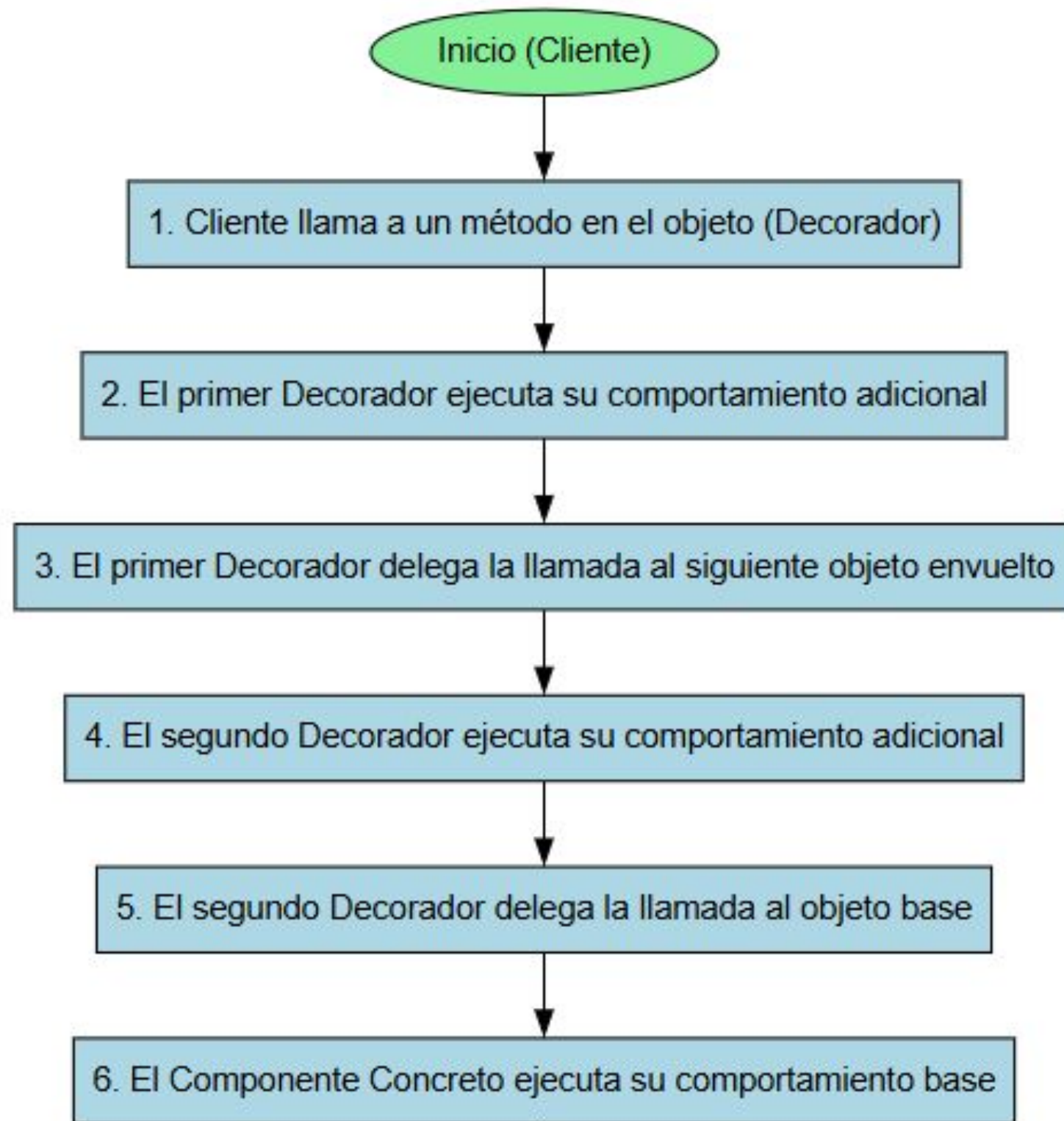
Decorator pattern – Diagram of sequence



CASO REAL

Mediante la implementación del patrón de diseño Decorator crearemos una aplicación que nos permite procesar un mensaje en capas, donde cada capa se encargará de procesar un mensaje a diferente nivel. primero convertiremos un Objeto en XML, seguido, lo envolveremos en un mensaje SOAP para después encriptar el mensaje, finalmente obtendremos un mensaje SOAP totalmente encriptado, el cual podrá ser enviado de forma segura a un destinatario. Cada capa de procesamiento será implementada con un decorador, y cada decorador podrá cambiar de posición para obtener un resultado diferente, de la misma manera, podrá ser agregados nuevos decoradores en medio de cualquier paso.





OTROS PATRONES

Flyweight

Flyweight es un patrón de diseño estructural que te permite mantener más objetos dentro de la cantidad disponible de RAM compartiendo las partes comunes del estado entre varios objetos en lugar de mantener toda la información en cada objeto.

Bridge

Bridge es un patrón de diseño estructural que te permite dividir una clase grande, o un grupo de clases estrechamente relacionadas, en dos jerarquías separadas (abstracción e implementación) que pueden desarrollarse independientemente la una de la otra.

Composite

Composite es un patrón de diseño estructural que te permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.

CONCEPTOS CLAVE APRENDIDOS

- Patron de diseño estructurales:
 - Adapter
 - Proxy
 - Facade
 - Decorator

EJEMPLO PRACTICO

Enunciado: Estás construyendo un sistema para una tienda de café en línea. Tienes una clase base Beverage con un método para obtener el costo. Necesitas añadir funcionalidades opcionales como Milk, Sugar, o Soy a cualquier bebida, sin modificar la clase base Beverage. Usa el patrón Decorator para crear un sistema que te permita combinar estas opciones de forma dinámica. Por ejemplo, un cliente debería poder pedir un Coffee simple, o un Coffee con Milk, o un Coffee con Milk y Sugar, y el sistema debería calcular el costo total de cada combinación

Practiquemos lo aprendido

Generar una analogía de aplicación del patrón adapter en un objeto de la vida real, identificando las diferentes partes que conforman la solución que implementa este patrón.



Nombre de la actividad:

Cantidad de participantes:

Doy fe que esta actividad está
planificada en dtt (Sí/No):

Hora de inicio:

Hora de fin:

Duración (min):

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)

Recursos

- <https://refactoring.guru/es/design-patterns/structural-patterns>



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA